

TELECOM NANCY - DEUXIÈME ANNÉE - 2018/2019

PROJET INDUSTRIEL

RAPPORT FINAL

Simulation du réseau de transport urbain par capsules Urbanloop

CHARLOTTE ASSELIN-BOULLÉ, BAPTISTE CHOCOT ET THIBAUT HENRY

Période du 18 Décembre 2018 au 20 Mars 2019

Projet réalisé en partenariat avec URBANLOOP



Encadrant industriel : Thibault CHOLEZ
pour Jean-Philippe MANGEOT et Philippe MANGIN

Encadrant académique : Thibault CHOLEZ

Déclaration sur l'honneur de non-plagiat

Nous soussignons,

Nom, prénom : Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT et Thibault HENRY

Élèves-ingénieur(e)s régulièrement inscrit(e)s en 3^e année à TELECOM Nancy

Numéro de carte des étudiant(e)s : 31618837 ,31622561, 31622481

Année universitaire : 2018-2019

Auteur(e)s du document, mémoire, rapport ou code informatique intitulé :

PROJET INDUSTRIEL, URBANLOOP routage Rapport Final

Par la présente, nous déclarons nous être informé(e)s sur les différentes formes de plagiat existantes et sur les techniques et normes de citation et référence.

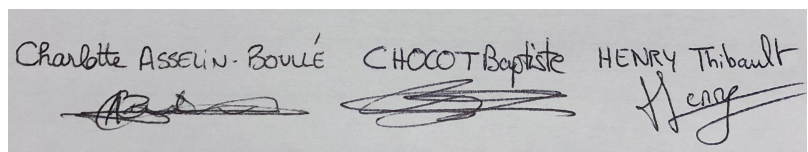
Nous déclarons en outre que le travail rendu est un travail original, issu de nos réflexions personnelles, et qu'il a été rédigé entièrement par nos soins. Nous affirmons n'avoir ni contrefait, ni falsifié, ni copié tout ou partie de l'œuvre d'autrui, en particulier texte ou code informatique, dans le but de nous l'accaparer.

Nous certifions donc que toutes formulations, idées, recherches, raisonnements, analyses, programmes, schémas ou autre créations, figurant dans le document et empruntés à un tiers, sont clairement signalés comme tels, selon les usages en vigueur.

Nous sommes conscient(e)s que le fait de ne pas citer une source ou de ne pas la citer clairement et complètement est constitutif de plagiat, que le plagiat est considéré comme une faute grave au sein de l'Université, et qu'en cas de manquement aux règles en la matière, nous encourrons des poursuites non seulement devant la commission de discipline de l'établissement mais également devant les tribunaux de la République Française.

Fait à Villers-Lès-Nancy, le 18 mars 2019

Signatures :



Charlotte ASSELIN-BOULLÉ CHOCOT Baptiste HENRY Thibault

Projet industriel

Rapport Final

Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT et Thibault HENRY

Année 2018-2019

Projet réalisé en partenariat avec URBANLOOP

Charlotte ASSELIN-BOULLÉ charlotte.asselin-boulle@telecomnancy.eu
Baptiste CHOCOT baptiste.chocot@telecomnancy.eu
Thibault HENRY thibault.henry@telecomnancy.eu

TELECOM Nancy
193 avenue Paul Muller,
BP 90172, VILLERS-LÈS-NANCY
+33 (0)3 83 68 26 00
contact@telecomnancy.eu

URBANLOOP
Projet dans le cadre du Pacte Grandes Ecoles regroupant la collaboration de 4 Écoles de Nancy:
l'ENSEM, l'ENSG, Les Mines de Nancy et TELECOM Nancy
<http://urbanloop.fr/>
Professeur coordinateur : Jean-Philippe MANGEOT
jean-philippe.mangeot@univ-lorraine.fr
Chef de Projet : Philippe MANGIN p.mangin@univ-lorraine.fr

Encadrant industriel : Thibault CHOLEZ

Encadrant académique : Thibault CHOLEZ thibault.cholez@inria.fr

Remerciements

Pour nous avoir offert la possibilité de travailler sur ce projet et de nous avoir fait confiance pour y contribuer, nous tenons à remercier Jean-Philippe MANGEOT et Philippe MANGIN et plus généralement l'ensemble des acteurs du projet URBANLOOP. Ce fut pour nous très formateur et une réelle chance de travailler sur un projet aussi innovant et complet.

Pour son encadrement et son aide précieuse et sans faille, nous tenons à remercier Thibault CHOLEZ, sans qui nous n'aurions pas autant appris ni pu réaliser notre projet.

Table des matières

Avant propos	1
Introduction	1
1 Conception du Projet	2
1.1 Contexte et présentation du projet	2
1.2 Spécification	2
1.3 Modélisation et choix techniques	4
1.3.1 Programmation Orientée Objet et découpage des classes	4
1.3.2 Notre simulateur : SimPy	4
1.4 Choix pour l'implémentation	6
1.4.1 Modélisation en graphes	6
1.4.2 Algorithme de routage	6
2 Réalisation	8
2.1 Présentation générale du projet réalisé	8
2.2 Simulateur	8
2.2.1 Retours a posteriori	8
2.2.2 Implémentation de la boucle événementielle	9
2.2.3 Les types événements	10
2.2.4 Trajet d'une capsule	10
2.2.5 Routage	11
2.2.6 Réalisations techniques	12
2.2.6.1 Discrétisation du temps et variations	12
2.2.6.2 Génération aléatoires des voyageurs	13
2.2.6.3 Affectations aléatoires des lieux de départ et de destination	14
2.2.6.4 Montée et descente des voyageurs	14
2.2.6.5 Affichage d'information en sortie standard	15
2.3 Interface graphique	16
2.3.1 Représentation du réseau	16
2.3.2 Panel d'interaction et d'informations	17
2.4 Difficultés rencontrées et choix effectués	18

2.4.1	Algorithme de routage	18
2.4.2	Construction de la représentation du réseau	19
2.4.3	Passage de données du simulateur vers l'interface graphique	20
2.4.4	Calcul de la position des capsules	20
3	Évaluation	22
3.1	Résultats et validation des objectifs	22
3.2	Améliorations apportées en plus des objectifs initiaux	23
3.3	Exploitation des résultats statistiques	23
3.4	Pistes d'améliorations	25
	Conclusion	27
	Annexes	29
	Étude de l'art	29
	Manuel d'utilisation de l'application	39
	Webographie	47
	Algorithme de Routage en pseudocode typé python	48
	Note de Cadrage et Compte-Rendus de réunions	51

Avant propos

Dans le cadre de la dernière année d'études à TELECOM Nancy, il est proposé aux futurs ingénieurs d'effectuer un projet à dimension industrielle : le Projet Industriel (susmentionné PI). Ce projet a pour vocation de confronter les étudiants par groupe de 3 ou 4 aux contraintes du monde de l'entreprise. Dans cette optique, différentes entreprises ou organisations proposent des sujets et un encadrement pédagogique.

Introduction

Dans le cas du projet concernant ce dossier, Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT et Thibault Henry ont été invités à collaborer au sein du projet URBANLOOP. Ce projet global s'inscrit dans le cadre d'un souhait de remaniement des transports urbains. L'objectif est de proposer un réseau cheminant des capsules individuelles lancées à grande vitesse dans des boucles interconnectées. Le projet ci-après présenté s'est concentré uniquement sur l'aspect de création d'une maquette numérique simulant ce réseau.

Il s'agissait principalement d'un projet de recherche mais dont l'enjeu était d'autant plus important qu'il devra servir de support pour la détermination de la faisabilité d'URBANLOOP. En outre, il servira de support aux futurs projets en permettant de pouvoir faire varier différents paramètres et étudier leurs influences. De plus, une attention particulière a été portée à la création d'une interface graphique, support visuel pour des présentations à l'avenir.

Pour faire état du travail réalisé, ce dossier comprend trois parties. Une première partie sur la conception qui regroupe le travail préliminaire qui a été effectué dans la première moitié du projet : présentation et choix de modélisations, choix des implémentations algorithmiques et choix techniques, le tout justifié. Dans une seconde partie nous traitons de notre réalisation : le projet abouti en le présentant d'un point de vue général mais aussi détaillé. Enfin, la dernière partie présentera notre évaluation : en marquant la validation des objectifs énoncés lors de la note de cadrage, mais aussi les points d'optimisation choisis et les limites qui donneront lieu à un listing des points à améliorer pour point de départ de futurs projets basés sur le nôtre.

1 Conception du Projet

S'agissant en grande partie d'un projet de Recherche et Conception, il a été convenu que la première partie du projet, se clôturant fin décembre, serait exclusivement consacrée à la recherche et conception théorique d'une solution. Cette phase est présentée dans cette partie. Il s'agit de présenter le plan d'action que nous avons défini, de déterminer les différents paramètres et moyens à mettre en œuvre, de choisir les technologies d'implémentation et de concevoir les premières ébauches des algorithmes principaux.

1.1 Contexte et présentation du projet

URBANLOOP s'inscrit dans le cadre d'un souhait de remaniement des transports urbains[1]. L'objectif est de proposer un réseau cheminant des capsules individuelles lancées à grande vitesse. Le projet global comprend une collaboration inter-écoles et une mise en place dans l'agglomération Nancéenne où un réseau de test verra le jour.

De multiples questions restent en suspens quant à la faisabilité du projet. Des études ont d'ores et déjà été effectuées sur les conditions de réalisation du projet, comme par exemple la technologie des roues par friction, ou encore la pose semi-enterrée des tubes qui dispose de nombreux avantages. Cependant, les principaux acteurs du projet ont besoin de connaître les valeurs optimales de certains paramètres mais aussi les valeurs limites et pouvoir jouer sur les configurations pour s'approcher au mieux d'un système le plus viable et satisfaisant possible. C'est l'objectif de notre projet.

Le projet ci-après présenté s'est concentré sur l'aspect de création d'une maquette numérique simulant le réseau, sans tenir compte outre mesure des autres projets collaborateurs. Le but était de prouver l'efficacité d'un réseau de type URBANLOOP par la paramétrisation de nombreuses variables définissant l'architecture du réseau (capacité totale, nombre de capsules par station, affluence d'une station, etc).

L'application finale devait donc simuler des capsules se déplaçant à vitesse constante sur un réseau de boucles et permettre de suivre l'évolution des capsules, l'état des stations de départ, et proposer quelques informations et statistiques plus globales (par exemple la congestion moyenne).

Pour bien réussir notre projet il s'agissait tout d'abord de bien le définir, ce que nous avons fait lors de la phase d'étude, jusqu'à mi-décembre.

1.2 Spécification

Pour déterminer les objectifs, nous avons en particulier listé les besoins que l'application aurait à satisfaire. Les besoins sont listés par ordre de priorité :

1. Réalisation d'une maquette numérique (ou simulateur) et conception de l'algorithme de routage des capsules optimisant le temps de transport. La maquette devra être réaliste avec visualisation unitaire des capsules et des usagers transitant en temps réel en fonction de plusieurs paramètres.
2. Modulation de la maquette numérique via différents paramètres à inclure. Le réseau

sera modélisé par un simulateur à événements discrets incluant :

- La topologie du réseau (nombre de boucles, d'interconnexions, de stations et de compartiments par station)
 - Les usagers : il faudra considérer une arrivée probabiliste des usagers avec des souhaits de trajets pseudo-aléatoires, avec la notion d'heure de pointe (une arrivée massive avec des destinations semblables).
 - Les différents délais : temps d'arrêt des capsules en station (montée et descente des usagers), temps d'insertion dans le réseau (correspondant au temps d'accélération).
 - La distance inter-capsules pour assurer la sécurité tout en exploitant au maximum le circuit.
 - La vitesse (constante) dans le circuit et la distance de sécurité inter-capsules.
 - Le nombre de capsules et d'usagers.
3. Inclure une interface de visualisation en "temps réel" et interactive.
 4. Tâche optionnelle : Créer un algorithme de stockage et d'approvisionnement des capsules pour anticiper les demandes des stations suivant les flux journaliers.

Nous avons aussi essayé de lister des paramètres à prendre en compte dans notre modèle :

- Le flux principal ;
- La capacité des stations ;
- Le modèle actuel accepte que la majorité des capsules dans le réseau soit vide, notre modèle pourra (et devra) donc être considéré de la sorte ;
- La file d'attente dans chaque station avant de rentrer dans une capsule ;
- La phase d'accélération (concentration sur la durée et non considération de la distance) ;
- L'existence de centres de stockage des capsules ;
- Gérer un flux important (heures de pointe) ;
- Gérer les intersections (changement de boucle d'une capsule et entrée dans le réseau) ;
- Possibilité de mettre en ligne le simulateur (optionnel).

Paramètres	Modélisation	Valeur
durée d'accélération	paramètre	6s
distance de garde	paramètre	50cm
vitesse	constante	80km/h
apparition des utilisateurs	fréquence variable avec trajets aléatoires	loi de poisson + gestion des heures de pointe
file d'attente en station	varie suivant les stations	provision à moduler (4 emplacements, minimum 2 capsules)
temps de sortie		utilisation d'une gaussienne centrée en 8s

Les différents paramètres pourront être variables mais une demande des informations sur la fréquentation actuelle des transports en commun a été faite aux organismes compétents [2] [3] et les résultats ont été pris en compte pour calibrer la maquette de la manière la plus réaliste possible.

Après avoir déterminé les aspects de modélisation, nous nous sommes intéressés aux questions de choix techniques.

1.3 Modélisation et choix techniques

1.3.1 Programmation Orientée Objet et découpage des classes

À la vue des différents éléments inter-connectés, nous avons choisi de nous orienter vers un langage de programmation orienté objet de manière à pouvoir modéliser les entités du réseau individuellement mais également les faire interagir facilement ensemble.

En prévision de l'implémentation des modèles de données, nous avons commencé à réfléchir à quoi devront ressembler les objets virtuels décrivant les objets physiques (capsules, stations, etc). L'objectif est de faire une description fidèle et complète des objets physiques pour que la simulation soit la plus fidèle possible à ce qui pourrait exister, en prenant en compte les contraintes imposées par les spécifications fournies par l'encadrant industriel. Notre modèle prévu a légèrement évolué au cours de notre implémentation et le fruit de notre réalisation est représenté figure 1 page 4.

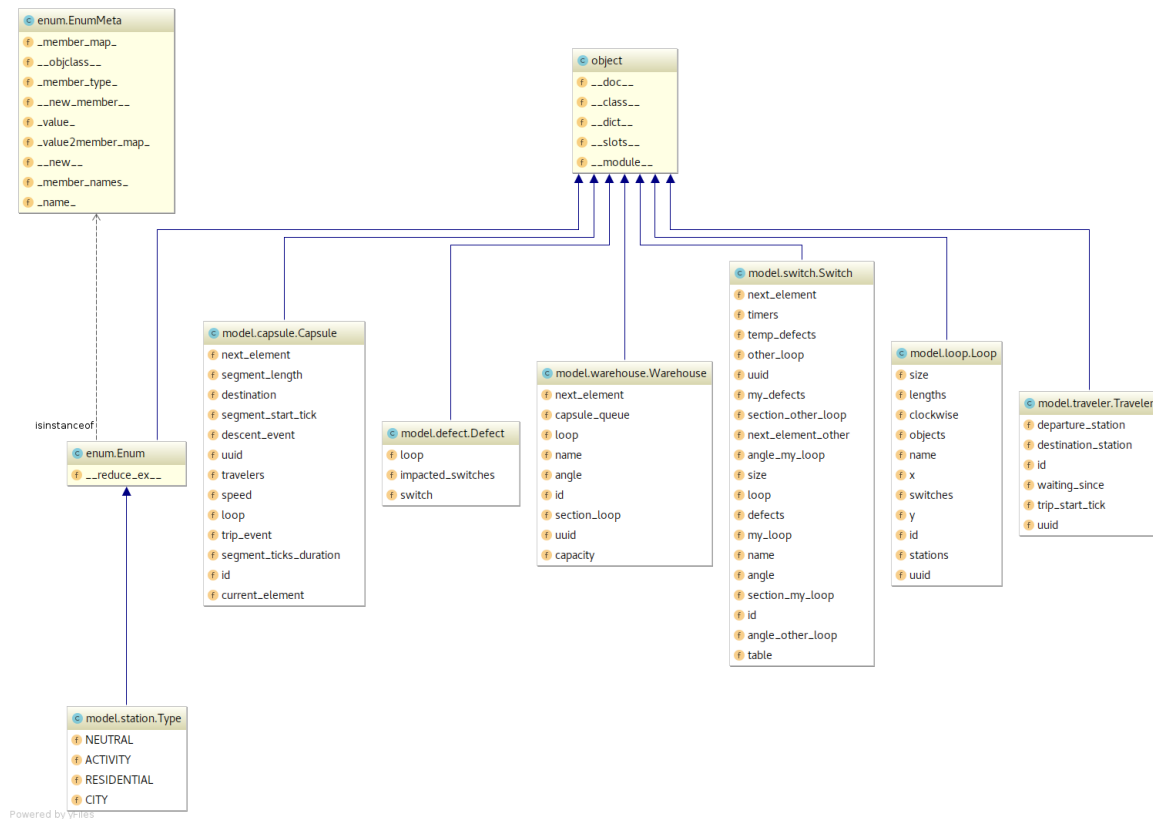


FIGURE 1 – Diagramme de classes correspondant au modèle de données implémenté

1.3.2 Notre simulateur : SimPy

L'objectif de notre projet est de prouver la faisabilité d'un réseau URBANLOOP en s'appuyant sur des résultats de simulation. La simulation du réseau est donc au cœur du projet. Thibault CHOLEZ nous a orienté vers la solution technologique des simulateurs à événements discrets. Il s'agit de modéliser le réseau et de discrétiser le temps, puis de générer des événements qui viennent à modifier l'état du système.

Nous avons donc cherché quels étaient les simulateurs à événements discrets les plus utilisés de nos jours, puis établi une liste de critères afin de trier sur le volet les différents choix qui s'offraient à nous. Nous souhaitions dans un premier temps une librairie libre de droit dans un langage qui supporte la programmation orientée objet. Nous avons isolé quatre candidats à évaluer plus en profondeur : DESMO-J[5], SystemC[6], SimJS[9] et SimPy[8]. L'étude de ces derniers est développée dans l'**annexe 1 - étude de l'art** [page 29].

Nous avons ensuite établi une liste de critères approfondie pour définir laquelle de ces quatre bibliothèques nous allions utiliser. Nous avons choisi des critères fonctionnels autour de la simulation (objectifs au maximum) ainsi que des critères non fonctionnels, plus larges, comme l'accès à de la documentation jusqu'à nos préférences et connaissances personnelles.

Au final, le simulateur qui a obtenu la meilleure note fut **SimPy**. Nous nous en sommes donc tenu à la notation et c'est celui-ci qui a été utilisé au cours de notre projet comme base de l'implémentation. Nous devions donc nous former à l'utilisation du langage Python durant les vacances de Noël pour combler nos lacunes. En outre, les premiers jours de la semaine bloquée de janvier pour le projet industriel ont été consacrés à la prise en main de ce simulateur.

En figure 2 vous retrouverez la notation en détail de tous les simulateurs :

Simulateurs		Desmo-J	SimJS	SimPy	SystemC
Critères					
Critères non fonctionnels					
Tutoriels / exemples	/2	0	1	2	1
Communauté active / Forums	/2	0	0	1	1,5
Documentation	/2	0	1	2	2
Langage de programmation connu	/2	2	1	1	0,5
Projets similaires déjà réalisés	/1	0	0	1	0
Licence libre	/1	1	1	1	1
Sous-total	/10	3	4	8	6
Critères fonctionnels					
Nombre d'objets simulés suffisant	/3	3	1	2	1
Possibilité d'utiliser une interface graphique	/3	2	3	3	3
Objets et fonctions prédéfinis adaptés	/3	2	2	1	2
Possibilité d'injecter un jeu de données	/3	1	0	1	1,5
Sous-total	/12	8	6	7	7,5
Total	/20	10	9	14	12

FIGURE 2 – Évaluation des bibliothèques suivant nos critères

En parallèle des aspects techniques nous avons déterminé certains éléments de l'aspect d'implémentation, en particulier des questions algorithmiques.

1.4 Choix pour l'implémentation

1.4.1 Modélisation en graphes

L'une des premières étapes de la spécification de notre simulation était d'abstraire la situation concrète du réseau URBANLOOP en un modèle mathématique simple mais le plus complet possible. Il fallait donc choisir une structure facilement modélisable mais dont l'abstraction pouvait être compréhensible par des personnes sans bagages mathématiques ou informatiques pointus mais nous permettant d'utiliser les fonctions et concepts mathématiques abstraits. URBANLOOP étant constitué d'un réseau de boucles qui contiennent différentes stations reliées entre elles par des axes, il a été très rapidement convenu d'utiliser les graphes orientés comme support. Ainsi nous avons pu définir que les sommets pouvaient être un objet de deux types : soit une station, soit un aiguillage, et les arêtes pouvaient être les tronçons de tubes entre deux stations ou entre une station et un aiguillage. Par ailleurs, la notion de poids sur une arête est associée à la distance parcourue puisqu'on veut actuellement minimiser le temps de trajet entre deux sommets. Cette fonction de poids pourra évoluer si on décide de limiter le nombre de changements de boucles (faisant ralentir la capsule) ou d'inclure d'autres paramètres de ce type.

Bien que ce ne soit pas encore utilisé, nous avons aussi introduit une seconde vision en graphe : celle considérant une boucle entière comme un sommet et les arêtes comme les aiguillages. Ceci nous permettra de déterminer les boucles possédant des stations associées à un puits ou une source en cas de panne d'un tronçon. Il s'agit intuitivement des boucles externes, généralement reliées par uniquement une liaison avec une autre boucle.

La figure 12 illustre la transformation d'un réseau simplifié en graphe : à gauche on a un dessin des stations et embranchements du réseau vu du dessus, et à droite le graphe correspondant. Ce modèle ne contient pas les distances et donc les poids sur les arêtes.

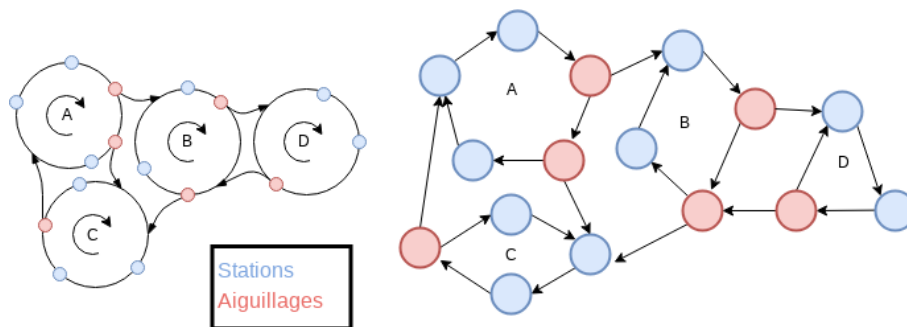


FIGURE 3 – Transformation d'un réseau simplifié en graphe orienté correspondant.

1.4.2 Algorithme de routage

Une des clés de ce projet résidait aussi dans l'algorithme de routage des capsules que nous devons choisir. Un algorithme de routage consiste à calculer le plus court chemin en termes de temps de trajet, dans un réseau donné.

Dans l'optique de définir l'algorithme de routage le plus adapté, nous avons réalisé un état de

l'art des algorithmes de routage (disponible dans l'**annexe 1 - étude de l'art** [page 29]) puis avons échangé entre nous pour déterminer une solution optimale en connaissance des solutions existantes. Nous avons ensuite mis au point une première ébauche de notre algorithme, que nous avons rédigé en pseudo-code et qui est disponible en annexe, *listing 1* page 48. Finalement la solution retenue est très proche de cette ébauche.

L'idée principale se base sur un algorithme de routage statique : une table par aiguillage dans laquelle on sauvegarde un chemin optimal calculé à la mise en route du réseau par un algorithme de Dijkstra. Cette table sera privilégiée : il s'agit de la **table permanente**. Cependant, pour pallier d'éventuelles anomalies pouvant intervenir durant la simulation (in s'agit "des défauts" que l'utilisateur doit ajouter manuellement, et qui correspondraient à un état de maintenance d'une section de boucle), un algorithme dynamique est mis en place. Il s'agit d'un calcul par état de liaisons ajusté à nos besoins. Chaque aiguillage a accès à la topologie du réseau et possède une table de routage qui lui est propre contenant un booléen lui indiquant s'il doit aiguiller ou non une capsule en fonction de sa boucle de destination.

Plus concrètement, à la construction initiale de la table permanente, on effectue un parcours de graphe en profondeur sous le concept d'état de liaisons puisque chaque aiguillage va chercher les informations sur les aiguillages adjacents itérativement. De ce fait, chaque aiguillage a une idée globale de la topologie générale du réseau et se construit un arbre de décision : sa table de routage. Ensuite, on ajoute une notion de communication entre les aiguillages pour qu'ils attestent qu'il n'y a pas de perturbation sur les deux tronçons qu'ils desservent. Nous avons d'ailleurs déterminé qu'il faudra ajouter un système de "durée de vie" des messages pour que la réception d'un ancien message ne perturbe pas le fonctionnement. Si un aiguillage ne reçoit pas le message d'un des autres aiguillages, ou s'il reçoit un message indiquant une perturbation sur une des boucles qu'il dessert, il recalculera en conséquence les trajets incluant l'aiguillage perturbé et la boucle concernée. Il est par ailleurs intéressant de voir que notre algorithme a été pensé pour gérer le fait qu'une boucle perturbée pourra être réajustée pour ne considérer la perturbation que sur un tronçon (ou section entre deux aiguillages, paralysant ainsi moins de stations par effet de bord. De même, les temporisateurs entre chaque envoi et attente de message pourront être réajustés, les changements du réseau ne devant être que ponctuels et exceptionnels (maintenances exceptionnelles ou réels problèmes techniques).

En conclusion, on se base sur une idée de routage dynamique mais avec un pré-enregistrement d'un chemin optimal à faire dans le cas général : il n'y aura à effectuer que des calculs partiellement et localement, en cas de perturbations (anomalies) seulement. En outre le routage se veut modulable : la partie étant indépendante il pourra être amélioré et modifié par la suite. On pourra rajouter un heuristique qui correspondra soit à l'orientation vers laquelle on doit se rendre (points cardinaux) ou bien une pondération sur le fait qu'il faille traverser un aiguillage (et donc limiter les insertions dans les boucles) ou encore le fait de devoir passer par des boucles centrales. Par exemple, aux heures de pointe, pour aller d'une station de périphérie à une autre il vaut mieux que la capsule fasse le tour plutôt qu'elle ne traverse Nancy.

2 Réalisation

2.1 Présentation générale du projet réalisé

Nous avons démarré la phase de conception à partir de fin décembre. Nous avons déjà déterminé le pseudo-code de l'algorithme de routage à implémenter, le langage de programmation avec la bibliothèque mettant en œuvre un simulateur à événements discrets, ainsi que l'architecture des objets à implémenter. Nous nous sommes réparti naturellement les tâches selon l'objet de nos recherches pendant la phase de conception précédente.

L'implémentation du modèle et la prise en main du simulateur furent relativement rapides. Nous avons réussi à obtenir dès fin janvier un simulateur basique d'un réseau URBANLOOP défini arbitrairement par nos soins, mais pas fantaisiste pour autant. Après avoir reçu la visite de M. Jean-Phillipe MANGIN dans nos locaux, nous avons vite saisi l'importance pour le projet d'aboutir à une interface graphique et pas seulement une simulation.

L'interface graphique a été plus ardue à réaliser que prévu mais nos efforts ont finalement porté leurs fruits. Nous avons réussi à ajuster nos algorithmes en visualisant parfois des incohérences dans le comportement des capsules ou encore des aiguillages. Nous avons abouti à une vue du réseau qui - nous estimons - est largement acceptable pour des industriels, interactive et riche d'informations.

Pour exploiter la simulation, des statistiques vont devoir être générés pour valider ou non les modèles injectés via les paramètres de simulation. Le simulateur fournit actuellement une liste d'informations vis-à-vis des événements qui surviennent au cours de la simulation, ainsi que des quantités recensées périodiquement pour établir des graphiques (notamment des boîtes à moustaches). Les statistiques ne sont actuellement pas calculées directement mais des scripts basiques suffiront largement à l'exploitation des données que le simulateur fournit.

2.2 Simulateur

Comme nous l'avons évoqué précédemment et suite à notre étude sur les différents simulateurs à événements discrets, SimPy est apparu comme un choix évident et pertinent pour réaliser notre projet industriel. Cette partie du rapport portera ainsi sur les retours a posteriori concernant ce choix, sur l'implémentation du simulateur en lui-même, mais également sur la discrétisation événementielle du trajet d'une capsule sur le réseau simulé. Enfin, nous listerons et décrirons les multiples réalisations techniques implémentées dans notre projet.

2.2.1 Retours a posteriori

Le choix du simulateur n'a pas été pris inconsidérément et différents critères avaient été analysés avant cette décision (voir figure 2).

Après implémentation, le premier retour à effectuer est que notre choix s'est révélé être un succès. En effet, la documentation disponible sur le site officiel de SimPy fut un élément clé dans notre compréhension du fonctionnement de cette bibliothèque au cours des premiers jours

de la semaine bloquée de janvier. À travers les différents exemples explicités sur le site, nous avons pu facilement transposer et adapter nos intentions de simulation. Des fonctionnalités intéressantes comme la possibilité de lancer la simulation en temps réel, ou la possibilité de définir un événement de terminaison de la simulation, nous ont ouvert de nombreuses portes quant à la paramétrisation de la simulation et de l'interface graphique.

Le second retour est que nous n'avons pas exploré les possibilités de SimPy dans son intégralité. Pour élaborer nos différents algorithmes et processus, nous nous sommes contenté des principes de base de la bibliothèque : ce choix est controversé car sans doute certains points de développement auraient pu être abordés de manière différente et plus efficace, mais cette utilisation en surface nous a surtout permis de rester flexible quant à l'implémentation des objectifs de la note de cadrage. De plus, si à l'avenir ce projet tend à être redévelopper, la prise en main n'y sera que plus facile.

Le choix du langage Python a lui aussi été un réel succès. La flexibilité du langage a rendu plus facile la conception de nos algorithmes comme par exemple à travers l'utilisation des dictionnaires pour le chargement du fichier JSON concernant la topologie du réseau. La modélisation du réseau est rendue possible grâce la dimension orientée objet de Python. L'existence de nombreux framework Python open-source de développement web, comme *Flask*, nous a permis d'interfacer notre simulation avec une application web. Pour finir et toujours dans l'optique d'un projet à long terme, la facilité d'apprentissage du langage ouvre la voie à de futurs projets industriels ou PIDR.

Enfin l'algorithme de routage fut lui aussi pertinent puisque nous nous retrouvons à avoir une minimisation des distances parcourues en défavorisant la traversée d'un aiguillage (qui suppose une insertion). Les routes sont donc calculées automatiquement pour chaque trajet avec une adaptation dynamique suivant l'état du réseau.

2.2.2 Implémentation de la boucle événementielle

Pour pouvoir effectuer une simulation à l'aide de SimPy, nous avons besoin d'un processus central regroupant l'exécution des différents événements se produisant sur le réseau. Ce processus central se présente sous la forme d'une boucle infinie dont chaque tour représente une unité de temps. Durant cette unité de temps, un ou plusieurs événements peuvent se produire. La discrétisation du temps est un élément essentiel dans le simulateur et sera abordée dans le paragraphe correspondant. Cette boucle infinie ne l'est pas réellement car c'est l'environnement SimPy qui s'occupera de sa terminaison, de deux manières possibles :

1. Soit on indique au simulateur un nombre de tour de boucle (aussi appelé le nombre de ticks), qui, multiplié par la valeur temporelle d'un tour de boucle, représentera la durée de simulation.

```
1 simpy_env.run(until=int(config.sim['duration']))  
2
```

2. Soit on indique au simulateur un événement de terminaison qui à son exécution mettra fin à la boucle infinie.

```
1 simpy_env.run(_endless_quit_event)  
2
```

Chacun des processus doit être enregistré dans l’environnement SimPy avant de pouvoir être déclenché par le simulateur et produire des événements. Il en va de même pour notre boucle événementielle centrale, appelée `sim_loop.loop()`, qui elle enregistre des processus à chaque tour. En revanche, il faut qu’elle soit enregistrée dans le simulateur avant son lancement, car une simulation sans processus enregistré s’arrête instantanément avec la levée d’une exception.

À chaque tour de boucle, un compteur est incrémenté pour pouvoir connaître le numéro du tick courant et donc pouvoir connaître l’ordonnancement des processus. Ce numéro est appelé `current_tick`.

2.2.3 Les types événements

Pour qu’un processus (ou fonction) puisse être validé et interprété par l’environnement SimPy, il faut qu’il génère un événement. À défaut de générer un événement, une exception sera levée par le simulateur. De plus pour être pris en compte, il faut que l’événement généré soit dans l’état *triggered*, i.e déclenché, qui se subdivise en deux états, succès ou échec. Dans le cas où un processus génère un événement non déclenché, rien ne se produira : cela permet d’élaborer des processus conditionnels qui ne déclenchent des événements que sous certaines conditions.

Durant le développement des fonctionnalités de la simulation du réseau URBANLOOP, l’utilisation des différents types d’événements est restée basique. Toutefois, les deux types d’événements utilisés sont largement suffisants pour répondre à nos besoins techniques. Il s’agit de :

- L’événement de base : c’est un événement sans particularité spécifique, qui doit être déclenché manuellement via les fonctions `success()` ou `fail()`.
- L’événement *Timeout* : c’est un événement qui se déclenche automatiquement. L’avantage de cet événement est qu’il est possible d’y attribuer des *callbacks* (ou fonctions rétroactives), qui s’exécuteront après un certain nombre de ticks renseigné :

```
1 ascent_timeout = sim_loop.get_env().timeout(10)
2 ascent_timeout.callbacks.append(lambda event: capsule.start_trip())
3
```

Ainsi, en exploitant ces deux types d’événement, l’ensemble des éléments techniques bloquants qui se présentaient à nous a pu être surmonté. Dans les paragraphes suivants, nous détaillerons comment nous avons réussi à simuler le trajet d’une capsule puis nous détaillerons les nombreuses réalisations techniques développées durant les trois derniers mois. Quant à la description des difficultés, elles seront abordées dans le paragraphe dédié 2.4.

2.2.4 Trajet d’une capsule

Pour pouvoir comprendre comment la simulation du trajet d’une capsule a été réalisée, il faut au préalable connaître la modélisation effective de la classe Capsule dans le projet. En effet, depuis les premières ébauches de diagramme de classes, certains de nos modèles ont été légèrement modifiés et/ou optimisés : celui des capsules en fait partie. Dorénavant, les attributs `current_station` et `next_station` sont respectivement devenus `current_element` et `next_element`

car nous nécessitions une plus grande précision concernant la position courante de la capsule sur le réseau. La notion d'élément englobe les trois structures se trouvant sur les boucles, à savoir les stations, les switches et les entrepôts. À chaque instant nous connaissons donc l'élément courant sur lequel se situe une capsule, sa destination finale et le prochain élément à atteindre dans le but de terminer le trajet défini par l'algorithme de routage. Partant de ce postulat, nous avons pu lier la gestion du trajet de la capsule par un algorithme récursif dont voici les étapes :

- Départ de la capsule de la station initiale après la montée du voyageur.
- Lancement du processus *update_trip()*. Celui-ci va déterminer la distance séparant les éléments courant et suivant. Ensuite, nous calculons le temps nécessaire pour que la capsule parcoure cette distance à vitesse constante (définie dans les paramètres de la simulation). Ce temps en seconde est converti en nombre de ticks (on rappelle qu'un tick correspond à un tour de la boucle centrale de simulation) pour se conformer aux besoins de SimPy et permettre la création d'un événement de type timeout avec comme paramètre le nombre de ticks calculé. Nous affectons pour terminer une fonction rétroactive *callback_trip_event()*, qui sera exécutée après l'écoulement du timeout.
- Ce callback constitue l'étape suivante. En effet, à la fin du timeout, la capsule a théoriquement parcouru la distance qui la séparait de l'élément suivant : nous pouvons alors actualiser les éléments courant et suivant. Pour ce qui est de l'élément courant, il prend tout simplement la valeur de l'élément suivant. En revanche, pour l'actualisation de l'objet suivant dépend du type d'objet qu'est l'élément sur lequel la capsule arrive :
 - Si c'est une station, l'élément suivant de la capsule deviendra l'élément suivant de la station.
 - Si c'est un switch, il est possible que la capsule ait besoin de changer de boucle, c'est-à-dire de traverser le switch en question selon le trajet défini par l'algorithme de routage. Si c'est le cas, l'élément suivant de la capsule deviendra l'élément suivant du switch sur la nouvelle boucle, sinon il deviendra l'élément suivant du switch sur la boucle actuelle.
- Après avoir actualisé les éléments courants et suivant, il faut vérifier si la capsule est arrivée à destination. Si tel est le cas, un processus de fin de trajet sera enregistré dans la boucle événementielle, et s'occupera de conclure le trajet par la descente du voyageur (voir le paragraphe dédié 2.2.6.4), ainsi que l'ajout de la capsule dans la file d'attente de la station destination. Au contraire, si la destination n'est pas atteinte, l'algorithme sera récursivement relancé par l'enregistrement et le lancement du processus *update_trip()* vu à la deuxième étape de cette explication.

Il faut également noter que l'implémentation d'algorithmes anticollision était initialement prévue mais n'a pas pu être réalisée dans les temps impartis. À l'heure actuelle, plusieurs capsules peuvent sortir d'une station en simultanée (et donc avec collision), ce qui n'est pas réaliste. Les capsules se déplaçant à vitesse constante sur notre simulation de réseau, seuls deux points sont à solutionner pour éviter les collisions : le départ des stations évoqué à l'instant, et le croisement probable de capsule à la sortie d'un switch.

2.2.5 Routage

Le choix de la route pour aller à une destination donnée est effectué automatiquement par le simulateur. En effet, le réseau des boucles, maillés par les aiguillages, permet un calcul du

“plus court chemin” par un algorithme de Dijkstra. Chaque aiguillage sait donc, en fonction de la destination d’une capsule qui arrive à son niveau, s’il doit ou non aiguiller la capsule vers l’autre boucle. Le coût d’un trajet qui est minimisé est calculé suivant la distance au sein d’une boucle et en considérant un coût plus élevé s’il faut traverser un aiguillage (ce coût supplémentaire est représenté graphiquement par l’interconnexion entre deux boucles, auquel peut être rajouter une constante configurable). Le routage initial est statique mais lorsqu’il y a une anomalie que l’utilisateur ajoute, le “nouveau chemin optimal” est calculé en limitant les passages par la section entre deux aiguillages qui est concernés. Dans la réalité ceci correspondrait à un besoin de ne pas passer par une section de boucle pour maintenance ou si l’arc est saturé.

En conséquence lorsqu’une capsule a pour *next_element* un aiguillage (switch) elle lui indique sa station de destination, laquelle est associé à la section de boucle dans laquelle elle est présente. Le switch consulte donc la table de cette section et peut donc savoir si la capsule doit ou ne doit pas changer de boucle.

2.2.6 Réalisations techniques

2.2.6.1 Discrétisation du temps et variations

La principale difficulté lorsque l’on utilise un simulateur à événements discrets est que la notion d’écoulement du temps est très floue voire complètement inexistante dans certains cas. Or dans notre situation, nous aimerions garder une proximité avec la temporalité pour répondre à des besoins évidents. Par exemple, lorsque les utilisateurs de l’application web souhaitent définir le temps de montée ou de descente d’un voyageur dans une capsule, il est logique de penser qu’ils le renseigneront avec comme unité la seconde. La question alors soulevée est de savoir mettre en relation le déroulement de la simulation avec une représentation temporelle. Il existe principalement deux manières de traduire l’exécution de la boucle événementielle en valeur temporelle.

La première qui n’est pas celle choisie, consiste à déterminer un nombre fixe d’événements représentant une seconde. Le temps y serait donc discrétisé par un nombre constant d’événements se produisant temporellement parlant à la suite les uns les autres : puisqu’un événement représente une fraction de seconde, deux événements ne peuvent pas avoir la même temporalité d’exécution. Cette option n’est pas exploitable dans notre cas puisque l’exécution simultanée d’événements est primordiale. Par exemple, deux capsules dans deux stations différentes doivent pouvoir démarrer leur trajet respectif au même instant.

La seconde option que nous avons choisie consiste à attribuer une valeur temporelle à l’exécution d’un tour de boucle, ou tick : nous avons désigné cette valeur par le nom de *sim_tick*. Ainsi, puisque nous pouvons déclencher autant d’événements que l’ordinateur peut en calculer, cette solution nous permet à la fois d’obtenir une traduction temporelle du nombre de ticks, et d’exécuter plusieurs événements en simultané. Pour revenir à l’exemple évoqué précédemment, si un utilisateur souhaite définir un temps de montée de 8 secondes, cela représentera $(8 / \text{sim_tick})$ nombre de ticks.

Par défaut, nous avons fixé la valeur du *sim_tick* à 0.05 secondes. Il est important de noter que plus cette valeur sera élevée et moins la simulation sera précise quant à son réalisme global et quant à l’exécution simultanée d’événements. Sur l’interface graphique que nous avons développée, il est possible de faire varier la valeur du *sim_tick* : c’est ce que l’on a appelé le *jerky*

mode, c'est-à-dire mode saccade. L'augmentation de cette valeur augmente par conséquent le nombre de secondes qui s'écoule à chaque tour de boucle. Cette variation est enregistrée dans une liste de tuples sous la forme suivante : (start_tick, tick_duration, sim_tick). En enregistrant les variations du sim_tick au cours de la simulation, on peut grâce à des calculs (peu informatifs pour ce rapport), obtenir le temps total simulé depuis le début de la simulation de manière correcte et fiable à chaque instant.

2.2.6.2 Génération aléatoires des voyageurs

Pour pouvoir générer des voyageurs de manière aléatoire et réaliste, nous avons implémenté une loi de Poisson à deux paramètres. Le premier correspond au nombre moyen de voyageurs empruntant le réseau chaque jour. Selon les dernières statistiques de la métropole du Grand Nancy, environ 95 269 personnes utilisent les transports en commun chaque jour¹. Le second paramètre et pas des moindres, concerne la fluctuation du trafic en fonction des heures de la journée. Comme vous pouvez l'observer sur la figure 4, en fonction de l'heure à laquelle nous voulons générer un nombre aléatoire de voyageurs, le résultat devrait varier fortement.

L'objectif est donc le suivant :

1. En moyenne sur une journée, nous aimerions observer empiriquement un nombre de voyageurs proche de celui indiqué en paramètre d'entrée.
2. Grâce à la temporalité du simulateur établie, nous aimerions faire varier le nombre de voyageurs par tranche horaire, de manière approximativement semblable au graphique ci-dessous.

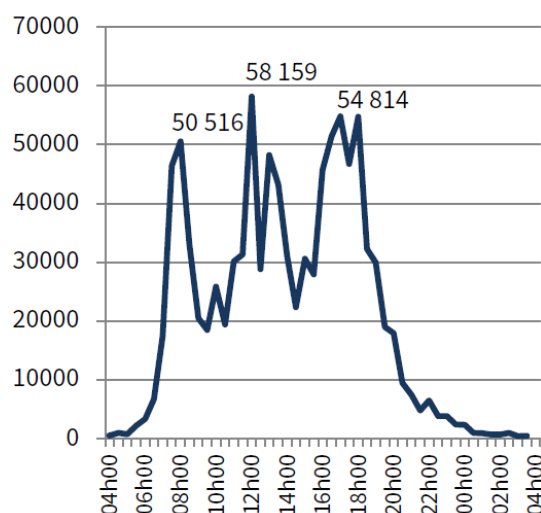


FIGURE 4 – Variation des déplacements au cours de la journée dans la métropole

Pour répondre à cette problématique, nous avons élaboré un modèle simpliste se basant un tableau de coefficient représentant la variation horaire du nombre de voyageurs sur le réseau des transports en commun. Ce tableau de coefficient, une fois intégré minutieusement dans le calcul de la loi de Poisson, nous a permis d'obtenir des résultats très satisfaisants et en cohérence parfaite avec nos objectifs. Sur la figure 5, vous retrouverez les totaux moyens sur un échantillon de 100 journées simulées, avec comme donnée d'entrée un nombre de voyageurs par jour de 4500.

1. Source : Enquête - Déplacement Sud Meurthe-et-Moselle

Heures	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	Total
Total	44,3	44,7	44,3	45,9	89,9	135	133	265	354	355	314	175	224	219	177	177	269	310	353	266	176	133	89,8	89,1	4483,3

FIGURE 5 – Variation empirique du nombre de voyageurs par heure

2.2.6.3 Affectations aléatoires des lieux de départ et de destination

Pour affecter aléatoirement un lieu de départ et un lieu de destination à chaque voyageur généré, nous avons défini pour chaque station un type représentant son intérêt principal. Ainsi, trois types de station sont possibles dans notre simulation :

1. La zone urbaine - *Type City* : une zone de ce type représente une station d'un centre-ville, ou d'une zone où la population environnante est dense. Les stations City auront une attractivité constante au cours de la journée, aussi bien dans le sens des départs que dans le sens des arrivées.
2. La zone d'activité - *Type Activity* : une zone de ce type représente une station proche des lieux de travail ou d'étude, comme une université ou une zone industrielle. Pour les stations Activity, dans le sens départ, il y aura une attractivité en baisse le matin et en hausse le soir. Dans le sens des arrivées, il y aura une hausse le matin et une baisse le soir.
3. La zone résidentielle - *Type Residential* : une zone de ce type représente une station proche des lieux fortement résidentiels comme des complexes HLM ou des résidences universitaires. L'attractivité est l'inverse de l'attractivité des zones d'activité.

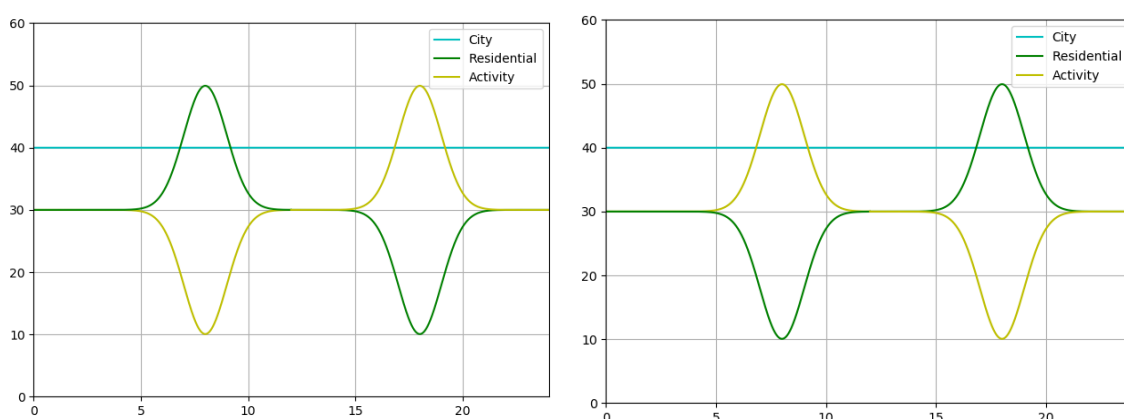


FIGURE 6 – Probabilités d'affectation par heure (départs à gauche, arrivées à droite)

Vous pouvez observer sur la figure 6 que les probabilités des zones résidentielles et d'activité sont horizontalement symétriques. En effet, elles sont construites à partir d'une valeur commune, et d'une valeur de fluctuation les influencera positivement ou négativement selon le cas lors des heures de pointes du matin et du soir.

Au niveau de la configuration, l'utilisateur pourra paramétrer les valeurs des probabilités du type *City*, et la valeur commune aux types *Residential* et *Activity*. Enfin, la fluctuation Activité-Résidentielle et les heures de pointes du matin et du soir sont également paramétrables : sur le graphe en figure 6, la fluctuation Activité-Résidence est de 20%, et les heures de pointes ont été fixées à 8h et 18h.

2.2.6.4 Montée et descente des voyageurs

Pour la montée et descente des voyageurs, nous avons donné la possibilité à l'utilisateur de configurer la durée moyenne que prendra un passager pour s'introduire ou s'extirper de la capsule. La durée effective appliquée sera légèrement aléatoire par rapport à la valeur moyenne fixée dans la configuration, avec une courbe gaussienne.

En ce qui concerne l'exécution du processus de montée des voyageurs du réseau dans les capsules, il est enregistré à chaque tour de la boucle centrale de simulation. Ce processus est conditionnel puisqu'il ne déclenche pas d'événements à chaque exécution : en effet, pour qu'un événement « montée d'un voyageur dans une capsule » soit déclenché, il faut tester pour chaque station si une capsule et un voyageur sont disponibles pour le départ. En fonction de cette condition, la capsule pourra ou ne pourra pas démarrer son trajet.

En ce qui concerne l'exécution du processus de sortie des voyageurs des capsules, celui-ci est enregistré lorsque que la capsule atteint sa destination finale. Il se présente sous la forme d'un événement timeout dont la durée est celle explicitée précédemment : cela signifie que cet événement est toujours déclenché et qu'à la fin de la période précisée, le voyageur sera effectivement descendu de la capsule.

2.2.6.5 Affichage d'information en sortie standard

```
[DEBUG][08:12:54][Gare->Switch n°2] Capsule 33 continues its trip to Campus_ARTEM
[DEBUG][08:12:54][Gare->Switch n°2] Capsule 19 continues its trip to TELECOM Nancy
[DEBUG][08:12:55][Switch n°4->Auchan] Capsule 35 continues its trip to Sapiniere
[DEBUG][08:12:55][Switch n°2->Switch n°1] Capsule 15 continues its trip to Campus_ARTEM
[INFO][08:12:55][Loop:Nancy] Capsule 15 stays on its loop
[DEBUG][08:12:56][Switch n°2->Switch n°1] Capsule 24 continues its trip to Velodrome
[INFO][08:12:56][Loop:Nancy] Capsule 24 stays on its loop
[DEBUG][08:12:56][Stanislas->Gare] Capsule 37 continues its trip to Gare
[INFO][08:12:56][Gare] Capsule 37 (Empty) ends its trip
[DEBUG][08:12:56][Gare] Contains 1 capsules now
[DEBUG][08:12:58][Switch n°4->Nancy Thermal] Capsule 11 continues its trip to Nancy Thermal
[INFO][08:12:58][Nancy Thermal] Capsule 11 (1 traveler) ends its trip
[DEBUG][08:12:58][Nancy Thermal] Contains 1 capsules now
[DEBUG][08:12:58] Start descent in capsule 11
[DEBUG][08:12:59][Switch n°2->Switch n°1] Capsule 16 continues its trip to Campus_ARTEM
```

FIGURE 7 – Illustration des différents sorties produits par le simulateur

Il y a différents événements générant des logs :

À titre informatif :

- lorsqu'un voyageur arrive dans la file d'attente d'une station lorsqu'un voyageur monte dans une capsule
- lorsqu'une capsule part d'une station
- lorsqu'une capsule passe par un aiguillage. Si elle reste sur sa boucle / change de boucle
- lorsqu'une capsule arrive dans une station
- lorsqu'un voyageur descend d'une capsule
- lorsqu'un défaut est généré
- si une station est pleine et donc ne peut pas accueillir une capsule dont ladite station était sa destination

À titre de debugging :

- chaque passage d'une capsule par une station, un entrepôt ou un switch lors de son trajet

- lorsque les processus de drainage/remplissage des stations sont lancés par les entrepôts
- les changements d'états du simulateur (vitesse/pause)

Ces éléments d'informations sont essentiels pour connaître le bon déroulement des opérations au cours de la simulation, mais également pour faire du debugging sur l'application web lorsque les comportements attendus ne se réalisent pas.

2.3 Interface graphique

2.3.1 Représentation du réseau

L'interface graphique se présente finalement sous la forme d'une page web, comme le montre la figure 8.

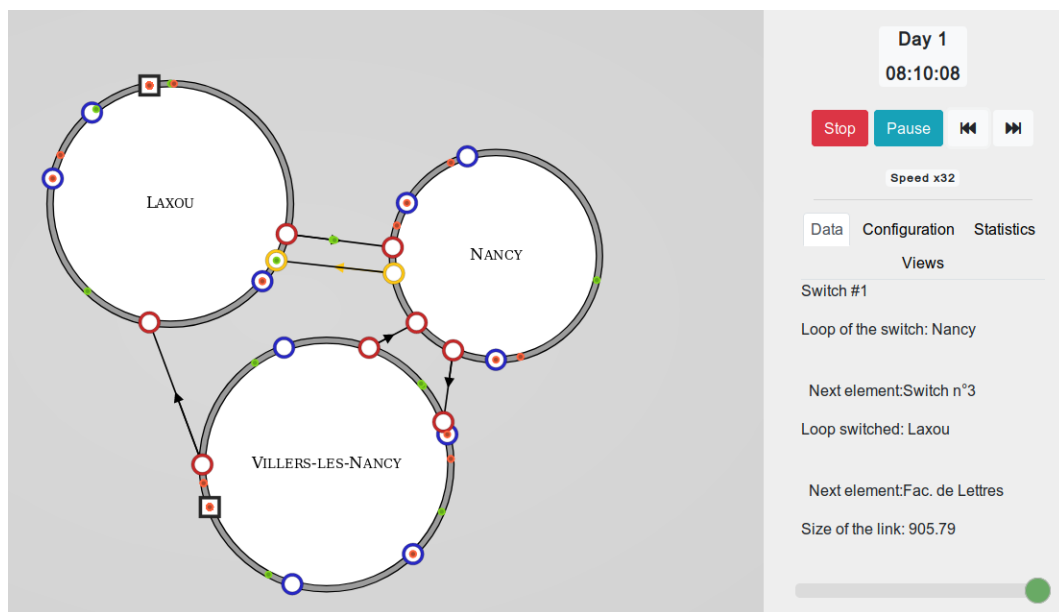


FIGURE 8 – Interface graphique du simulateur.

La majeure partie de la page correspond à la représentation du réseau URBANLOOP chargé. L'image centrée dans cette zone peut être déplacée, zoomée, et les objets représentés peuvent être grossis en cas de besoin. On distingue dans cette représentation plusieurs objets distincts :

- Les grands cercles gris : il s'agit des boucles. Elles structurent le réseau en accueillant en elles les stations, aiguillages et entrepôts. Les boucles sont unidirectionnelles.
- Les cercles bleus : il s'agit des stations. Elles accueillent des capsules et des passagers y sont générés. Elles fournissent des informations qui évoluent au fil du temps, comme le nombre de passagers en attente d'une capsule, ou encore le nombre de capsules disponibles actuellement, etc.
- Les cercles rouges : il s'agit des aiguillages. Ils sont représentés par paire (un pour l'entrée de l'aiguillage et un pour sa sortie) et sont unidirectionnels eux-aussi. Attention toutefois car il s'agit d'un seul et même objet virtuel en arrière-plan, d'où la sélection du couple lors d'un clic sur une entrée ou une sortie.
- les carrés sombres : il s'agit des entrepôts. Ce sont des zones où sont stockées des capsules en cas de besoin. Elles servent notamment à alimenter les stations vides de capsules, ou bien à accueillir des capsules quand certaines stations sont trop pleines.

- les petits disques verts et rouges : il s’agit des capsules. La couleur verte indique qu’un passager se trouve dedans. Elle affiche de nombreuses informations quant à leur état actuel (destination, passager, etc).

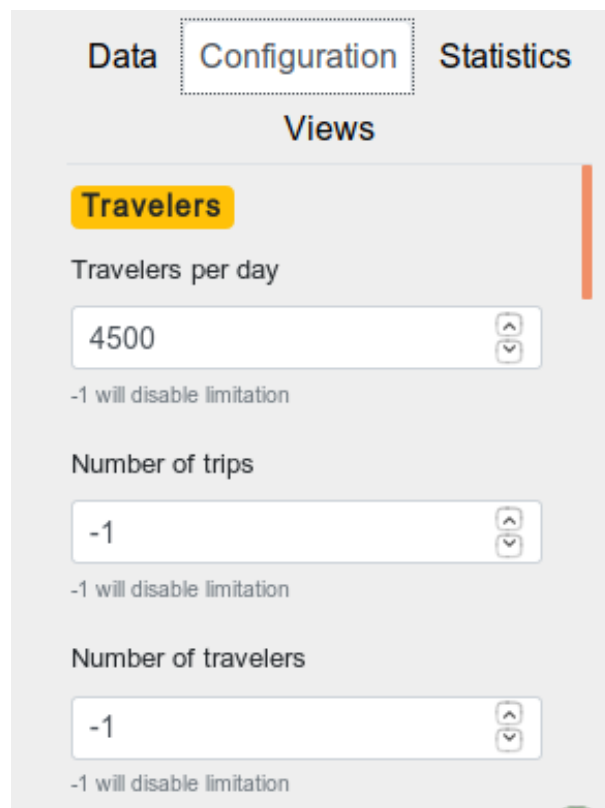
2.3.2 Panel d’interaction et d’informations

Sur la partie droite de l’interface, on retrouve des boutons permettant de lancer, mettre en suspens et d’arrêter la simulation. On peut aussi l’accélérer à l’aide des flèches de vitesse. Attention toutefois à ne pas abuser de l’accélération car à partir d’un certain seuil, la simulation passe en mode *jerky* : on saute des événements pour que le visuel accélère. De telles accélérations n’ont pas beaucoup de sens en termes de simulation.

La partie inférieure est consacrée à l’affichage des informations et des interactions avec l’utilisateur. On dispose de plusieurs choix comme l’affichage des informations, la modification de la configuration ou bien l’affichage des statistiques de simulation en temps réel.

On retrouve la possibilité de modifier la configuration en direct à l’aide du panel dédié, comme montré figure 9. Les paramètres de simulation sont triés par catégories, et influent directement la simulation après sauvegarde. On peut injecter directement différentes configurations pour tester le comportement et les limites de notre simulateur.

On peut également choisir de modifier temporairement la configuration : le fichier de configuration n’est pas affecté mais la simulation l’est. En cas de modifications excessives, un bouton permet de recharger la configuration par défaut (voir figure 10).



The image shows a screenshot of the 'Configuration' panel in a simulator interface. At the top, there are three tabs: 'Data', 'Configuration' (which is selected and highlighted with a dashed box), and 'Statistics'. Below the tabs is a section titled 'Views'. Under 'Views', there is a sub-section titled 'Travelers' in a yellow box. Below this, there are three configuration items, each with a text input field and up/down arrow buttons:

- Travelers per day**: The input field contains '4500'. Below it, a small grey text note says '-1 will disable limitation'.
- Number of trips**: The input field contains '-1'. Below it, a small grey text note says '-1 will disable limitation'.
- Number of travelers**: The input field contains '-1'. Below it, a small grey text note says '-1 will disable limitation'.

FIGURE 9 – Panel de configuration du simulateur.

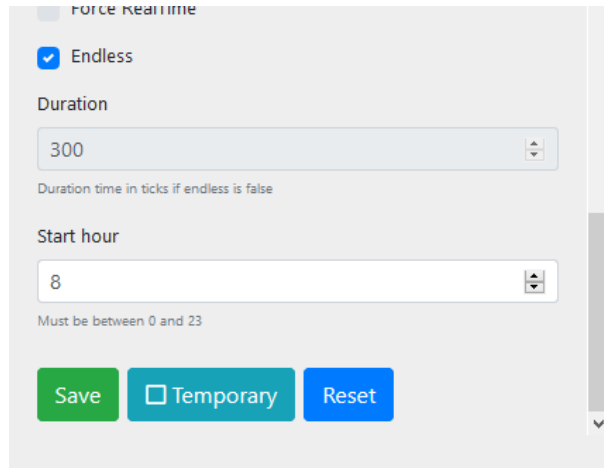


FIGURE 10 – Fin du panel de configuration du simulateur.

2.4 Difficultés rencontrées et choix effectués

2.4.1 Algorithme de routage

La difficulté de l'implémentation du système de routage que nous avons choisi était double. D'abord il fallait définir avec précision nos objets, et en particulier les tables de routage : ce qu'elles devaient contenir et la manière de le stocker. Nous avons opté pour des dictionnaires dont les clés sont noms des boucles (ou plus tard les sections de boucles). Il fallait ensuite faire en sorte que le calcul soit automatisé et itératif. Le second obstacle, bien plus imposant, était de gérer l'aspect dynamique : il fallait modifier toutes les tables de tous les switchs impactés, effectuer le calcul adéquat, puis pouvoir "revenir à l'état d'origine".

D'un point de vue plus détaillé nous avons eu quelques difficultés à tester l'algorithme de routage dans un premier temps car nous n'avions pas tout de suite intégré la simulation avec l'interface graphique. Au lancement de l'interface graphique, nous avons pu tester visuellement le comportement du routage et repérer d'éventuelles incohérences.

En outre nous avons choisi, après avoir longtemps fonctionné avec une version considérant les boucles en entières, de raisonner par section de boucle correspondant au tronçon entre 2 aiguillages de la boucle (ils sont à minima au nombre de 2 dans une boucle : un pour entrer et un pour sortir). Ce changement fut assez complexe à implémenter au vu de la diversité des cas possibles et de la non connaissance de l'emplacement exacte des stations depuis un switch.

Concernant l'aspect dynamique nous avons eu d'autant plus de difficultés une fois l'algorithme de routage par sections implémenté qu'il fallait pouvoir inclure des défauts dans le réseau et pouvoir analyser si les changements de routes étaient cohérents. Or l'ajout des défauts s'est fait très tardivement et beaucoup de changements ont dû être faits.

Par ailleurs l'aspect de défauts présuppose qu'il peut y avoir des destinations qui ne seront pas desservies tant que l'anomalie sera présente. Dans notre cas une capsule souhaitant se rendre dans une destination qui est sur une section inaccessible à cause d'un défaut bouclera indéfiniment jusqu'au retour à la table de routage initiale. Ce modèle n'étant pas réaliste il faudrait ajuster le fait que la capsule se dérouta vers la station accessible la plus proche de la destination voulue.

2.4.2 Construction de la représentation du réseau

L'interface graphique nous a donné beaucoup de fil à retordre. Nous avons dans un premier temps réalisé l'interface avec la librairie **Tk**, qui est native à Python et qui sert initialement à créer des fenêtres. Tous les objets étaient dessinés à l'aide des fonctions basiques fournies avec Tk, à savoir des fonctions servant à tracer des ellipses et des segments. Il a fallu définir la représentation de nos objets et de les tracer à l'aide d'outils rudimentaires. De plus, une fois le réseau dessiné, il fallait maintenant rendre interactif le dessin. Pour cela, à chaque clic de l'utilisateur, il fallait vérifier si la position du curseur lors du clic correspondait avec la position d'un objet dessiné. Cela fût globalement facile à réaliser : il suffisait de vérifier, pour chaque objet, si le clic avait eu lieu au-dessus de lui, en vérifiant en priorité les objets dessinés en dernier (ceux qui sont au premier plan). Cependant, un problème survint grâce à la topologie de notre réseau de test, à savoir que les bords supérieurs et gauches de notre dessin avaient des coordonnées négatives, comme le montre la figure 11.

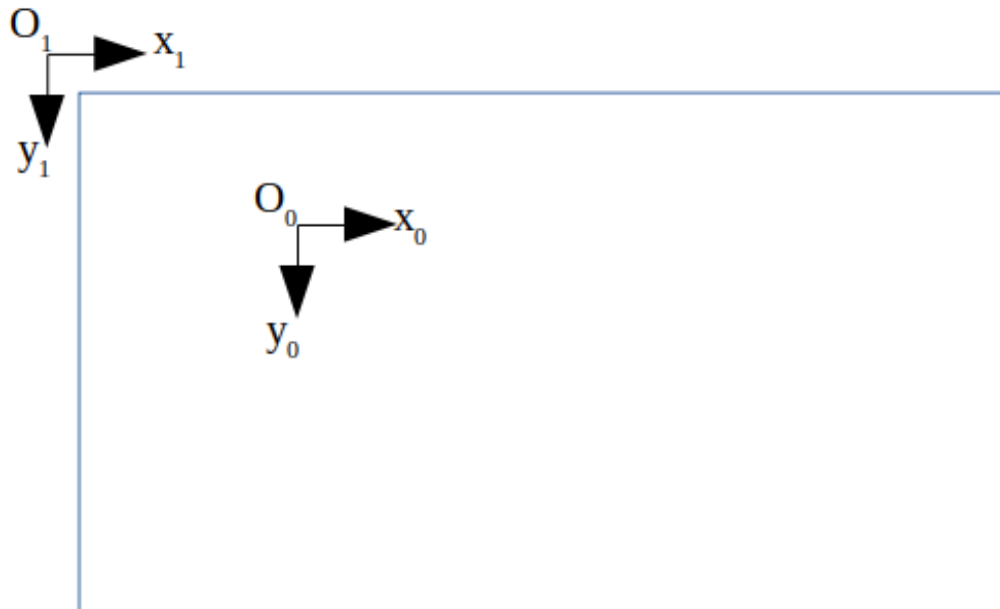


FIGURE 11 – Les différents repères lors de l’affichage du dessin.

Le cadre bleu représente la surface sur laquelle notre réseau était dessiné initialement. Le repère $R_0 = (O_0, x_0, y_0)$ est le repère propre au réseau, et le repère $R_1 = (O_1, x_1, y_1)$ est le repère propre à l’objet graphique contenant l’image. L’écart entre le cadre et le repère R1 était aléatoire et dépendait de la taille de la fenêtre ainsi que de la taille du réseau, ce qui rendait difficile d’identifier avec précision la position équivalente sur le dessin d’un clic. Illustrons par un exemple. Lorsque l’utilisateur cliquait sur un objet, les coordonnées (x, y) du clic étaient données dans le repère R1. Sans l’offset aléatoire entre le cadre et R1, la conversion était plutôt simple :

$$\begin{aligned} - x_{\text{dessin}} &= x - O_0.x - O_1.x \\ - y_{\text{dessin}} &= y - O_0.y - O_1.y \end{aligned}$$

Cependant le côté aléatoire de l’offset devait être ajusté pour chaque taille d’écran, et pour chaque réseau. De plus, à chaque sélection d’un élément, l’ensemble du dessin était à reconstruire car on ne pouvait que redessiner par-dessus le dessin actuel, comme avec une feuille de papier.

Un problème supplémentaire était qu'on ne pouvait pas (dé)zoomer sur l'image et qu'on ne pouvait pas la déplacer non plus. Nous avons essayé d'utiliser la classe **QWebView** proposé par la librairie Qt, initialement développée en C++ et qui a été copiée/collée en Python telle quelle. Nous espérions conserver l'interface telle quelle en affichant la représentation du réseau à travers une page web, intégrée au logiciel. Nos efforts furent vains et nous avons choisi de migrer vers une application web, à l'aide de la librairie **Flask**. Tous nos problèmes vis-à-vis de la représentation du réseau se sont résolus grâce à ce choix. Cependant cette décision a impliqué de recommencer une grande partie de l'implémentation de l'interface, et même si nous avons pu conserver des éléments, il a fallu refaire dans le nouveau langage le placement et la visualisation des objets (avec le lien avec le modèle du réseau chargé), la communication entre le modèle et la vue, la visualisation des données lors du clic, le panneau d'interaction, la fenêtre de choix des paramètres de configuration et celle du chargement des fichiers.

2.4.3 Passage de données du simulateur vers l'interface graphique

Lors du premier transfert de données depuis le simulateur vers l'interface graphique, nous avons deux problèmes majeures : le déplacement des capsules était discontinu (elles se "téléportaient" sur le réseau) et il arrivait que leur vitesse variait, ce qui était doublement incohérent avec notre modèle. À ce stade, l'interface lisait l'état du simulateur en temps réel, et nous avons soupçonné que ce comportement devait être modifié pour augmenter la cohérence du visuel au niveau des capsules. Après avoir échangé avec M. CHOLEZ et M. BOMBARDIER, nous avons imaginé de construire une liste d'états du simulateur en même temps que le simulateur générerait des événements, puis en parallèle de lire ces états périodiquement depuis la vue. Cependant, la liste d'états ne copiait pas les objets virtuels mais contenait seulement des références vers ces derniers. Cela a eu pour effet de ne pas du tout améliorer notre situation. Nous avons alors imaginé de copier périodiquement l'état de la simulation (concrètement à chaque modification) et de les stocker dans une queue d'événements à laquelle la vue avait accès. Le simulateur s'arrêterait automatiquement lorsque la queue était pleine, et redémarrerait lorsqu'elle était relativement vide (à chaque lecture par la vue, l'état affiché était retiré de la queue). Ce modèle fût extrêmement performant et c'est celui que nous avons conservé.

Pour ce qui était des problèmes de variation de vitesse, il s'agissait d'une erreur de conception lors de la traversée des aiguillages : nous n'avions pas encore implémenté la traversée progressive des aiguillages, ainsi la capsule se téléportait à la sortie de l'aiguillage, traversait le prochain segment aussi vite qu'elle devait traverser l'aiguillage (ce qui explique la différence de vitesse car les distances étaient différentes) et se re-téléportait ensuite à la station où elle devait arriver à la sortie de l'aiguillage (ce qui expliquait deux téléportations problématiques). Une fois la cause du problème identifiée, l'implémentation fût rapide et les problèmes identifiés fixés par la même occasion.

2.4.4 Calcul de la position des capsules

Le calcul de la position des capsules sur les boucles du réseau fut une tâche qui nous a consommé énormément de temps tout au long du projet. En effet, ce problème que l'on pensait associer à l'utilisation de la bibliothèque Qt s'est avéré être récurrent puisqu'il est également survenu lors du passage sur l'application web.

Lors de l’affichage des résultats du simulateur sur l’interface graphique, nous avons observé que sur les petits segments de boucle (un segment correspondant à la portion de boucle entre deux éléments), la vitesse des capsules était plus faible que celle sur les grands segments. Globalement, la vitesse des capsules était donc constante par segment et non constante sur l’entièreté du réseau.

Côté simulation, une capsule se "téléporte" d’une station à une autre (voir paragraphe sur le trajet des capsules). Le calcul de la position des capsules était donc primordial pour l’affichage graphique sur l’application web. Ce calcul a d’abord été modélisé sous la forme d’un pourcentage de trajet parcouru. Or après de nombreux calculs et vérifications, nous avons découvert que la distance parcourue par une capsule à chaque tick dépendait de la longueur du segment sur lequel elle se situait.

Pour résoudre le problème, nous avons donc dû calculer la distance absolue parcourue par une capsule à chaque tick en fonction de la vitesse de celle-ci au préalable indiquée dans la configuration. Cela a eu pour effet d’uniformiser la distance parcourue par les capsules à chaque tick et sur tous les segments du réseau. Par conséquent, cela a également permis de résoudre le problème de vitesse non constante.

3 Évaluation

Pour clore correctement l'implémentation de notre outil il a été nécessaire de faire le point sur ce que nous avons ou non validé, les optimisations qui vont au-delà des demandes de la note de cadrage et enfin faire l'inventaire des améliorations possibles à apporter au projet.

3.1 Résultats et validation des objectifs

Nous satisfaisons actuellement l'ensemble des critères demandés lors du lancement du projet et établis dans la note de cadrage (Disponible en annexe [page 54]). Dans ce sens nous avons demandé la programmation d'un test d'acceptation avec Philippe MANGIN et/ou Jean-Philippe MANGEOT, qui seront les principaux utilisateurs de notre outil.

En outre pour la validation de nos objectifs en cours de réalisation nous avons programmé durant cette seconde période de projet, différents points avec notre encadrant, monsieur Thibault CHOLEZ, qui est aussi l'intermédiaire gérant le projet URBANLOOP à l'échelle de TELECOM Nancy. Les comptes-rendus de ces points sont en annexes [pages 51-[page 64]].

Nous avons aussi effectué une rencontre globale de tous les acteurs du réseau le jeudi 7 Février pour expliquer notre apport dans le projet URBANLOOP. De même nous avons participé au tournage d'une vidéo de promotion avec le second groupe le mardi 6 Février. Nous avons par ailleurs une seconde vidéo de présentation prévue le mardi 20 Mars, jour de la soutenance qui marque la fin de notre projet. Ainsi, nous participons au rayonnement du projet URBANLOOP tout en validant les exigences des acteurs de ce projet.

Ainsi notre projet répond aux objectifs fixés comme suit :

1. Un simulateur effectif et réaliste.
 - Gère le flux principal (sur les boucles) : détermine les états.
 - Inclut dans le modèle le nombre des emplacements de capsules par stations (leur capacité).
 - Gère des files d'attente des usagers pour rentrer dans les capsules présentes ou arrivant dans la station.
 - Accepte la possibilité que des capsules vides soient dans le flux principal.
 - Nous ne gérons par contre pas la phase d'accélération pour s'insérer dans une boucle. un algorithme de routage optimisant la distance parcourue et donc (étant donnée la vitesse constante), le temps de transport.
2. Gestion de différents paramètres, dont certains configurables :
 - Le nombre de capsules et d'usagers paramétrables et déterminants.
 - Gère une arrivée probabiliste des usagers avec leurs trajets décidés pseudo aléatoirement (et rajout des notions d'heure de pointe qui implique une arrivée massive avec des destinations semblables)
 - Gère de différents délais (temps d'arrêts des capsules en station (montée et descente), temps d'attente en station, temps de trajet...
 - La distance de sécurité inter-capsules en maximisant l'exploitation du circuit n'a pas été pleinement respectée. En effet cette partie incombait au second groupe de PI avec lequel nous n'avons pas eu l'occasion de fusionner nos travaux. Ainsi, lors du départ des stations ou l'insertion dans une boucle depuis un switch, une collision est visuellement possible actuellement.

3. Inclure une interface de visualisation. Faire en sorte que cette dernière soit interactive.
 - La visualisation unitaire des capsules et des usagers transitants en temps réel. Bien que non unitaire dans les emplacements des stations sur l'interface.
 - Avoir accès aux données des objets
 - L'interface permet le changement des paramètres,
4. Nous avons aussi incluse la tâche optionnelle qui était de créer un début d'algorithme de stockage et d'approvisionnement des capsules pour anticiper les demandes des stations suivant les flux journaliers

Enfin pour appuyer l'utilisation de notre application nous avons écrit un manuel succinct qui présente les différentes fonctionnalités, ce dernier est présent en Annexe [page 39]. De plus, pour faciliter la reprise de notre projet, nous avons pris soin de documenter les fonctions et de commenter régulièrement notre code.

3.2 Améliorations apportées en plus des objectifs initiaux

En plus de valider les différents points impératifs, nous avons, comme expliqué dans les différents points de réalisation, effectuer des optimisations et des ajouts de fonctionnalités :

- La possibilité de cliquer sur les objets pour avoir accès aux informations le concernant.
- L'inclusion d'un modèles probabilistes avancés réalistes avec la notion de répartition horaire.
- La gestion de la répartition des capsules dans le réseau en fonction des demandes et capacité avec anticipation des besoins par heure.
- L'application visuelle déployable en ligne
- La récupération des statistiques principaux de la simulation qui permettra une exploitation des résultats numériques
- Une topologie de test réaliste en considérant des distances proches de la réalité.
- La gestion de la temporalité du simulateur. Initialement prévu comme ne devant être que en temps réel, nous avons pris le parti de pouvoir "accélérer" plus apprécier plus rapidement les différents paramètres.

-

3.3 Exploitation des résultats statistiques

Le simulateur permet également d'obtenir et d'extraire quelques informations sur la simulation. Certaines données considérées comme "ayant de la valeur" sont enregistrées au fur et à mesure de la simulation et sont retranscrites vers des fichiers lorsque la simulation s'achève. Les fichiers à considérer sont **latest.txt**, qui sera mis à jour à chaque nouvelle simulation, et un autre dont le nom dépendra de la date à laquelle la simulation se sera achevée (par exemple, **2019-03-18 08 :26.txt**). VOici une liste non exhaustive de statistiques relevées au cours d'une simulation :

- Le nombre de capsules quittant ou rejoignant une boucle ;
- Le nombre de capsules émises moyen par drainage ;
- Le temps d'attente moyen d'un passager ;
- Le nombre de départs de capsules d'une station ;

— etc.

Le simulateur génère deux types de statistiques. Certains statistiques caractérisent des évènements qui prennent place à des temps et lieu différents mais qui sont exactement identiques, par exemple, pour une boucle, l'évènement "une capsule quitte la boucle". D'autres varient et nécessitent d'enregistrer une valeur qui caractérise l'évènement, par exemple le temps d'attente moyen d'un voyageur dans une station pour obtenir une capsule. Ainsi, on retrouve dans les fichiers où sont enregistrées les informations deux types de lignes :

```
1 # Nombre de édparts de capsules d'une station .
2 #
3 # <class 'model.station.Station'>#1
4 16.05;95.05;133.05;394.8;499.0;518.65;587.15
```

Celle-ci représente le nombre de secondes auquel l'évènement prend place. On sait qu'à chacun de ces instants, une capsule a quitté la station 1.

```
1 # Nombre de capsules ééarrtes moyen d'une station .
2 #
3 # <class 'model.station.Station'>#0
4 0.0-4;30.0-4;60.0-4;90.0-4;120.0-4;150.0-3;180.0-2;210.0-2;240.0-2;270.0-1;300.0-0;
```

Ici, le couple [nombre de secondes - valeur] est enregistré. On ne caractérise pas ici un évènement qui se produit mais on enregistre un état avec sa valeur associée. En important ces différentes lignes dans un tableur, on peut en extraire des courbes pour évaluer le comportement de la simulation. Voici un petit exemple de courbes montrant le début d'une journée dans une station :

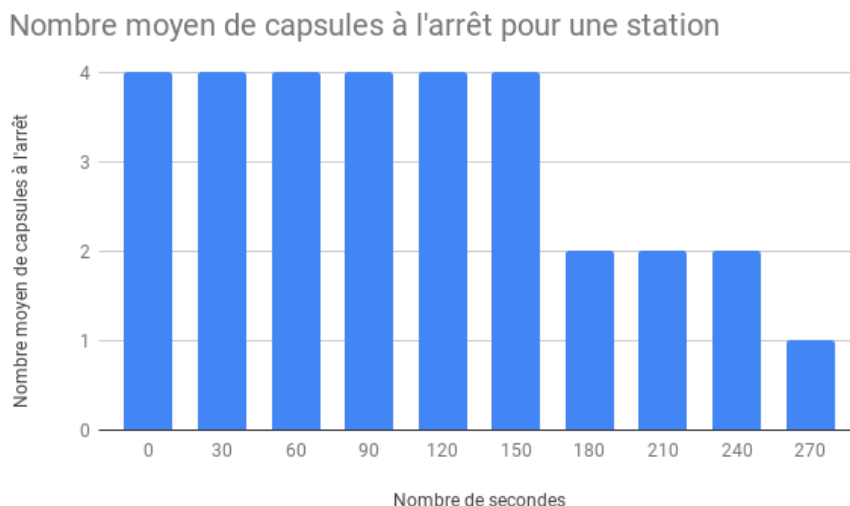


FIGURE 12 – Nombre de capsules disponibles dans une station exemple.

Le nombre d'informations enregistrées est encore faible aujourd'hui mais augmentable à l'infini. Plus le simulateur se verra complété, plus le nombre d'informations à enregistrer grandira. On peut d'ores et déjà tracer quelques courbes ou bien des diagrammes-moustaches afin d'évaluer le comportement des simulations en faisant varier les paramètres.

3.4 Pistes d'améliorations

Notre simulateur se veut complet et réaliste pour correspondre au mieux aux attentes et à la réalité de manière à pouvoir anticiper des éléments. Cependant, il reste certaines limites à notre solution. Nous avons, au fur et à mesure, établi une liste des possibles améliorations que nous aurions aimé faire ou les prochaines étapes qui sont envisagées. Cette liste peut servir de point de départ à la reprise de notre projet :

- D'un point de vue graphique il n'est pas encore possible d'ajouter des défauts pertinents via l'interface graphique au jour de rédaction du rapport. Cette fonctionnalité est prévue d'être ajoutée d'ici le test d'acceptation et/ou du rendu final du 20 mars. Il pourrait être intéressant de pouvoir ajouter manuellement depuis l'interface des voyageurs en spécifiant les stations de départs et d'arrivée.
- De même que le point précédent, la visualisation des emplacements des stations contenant ou non des capsules, et ce individuellement, n'est pas encore opérationnelle à ce jour. Là encore il est prévu que la correction ait lieu.
- Gérer la distance inter-capsules et donc la collision lors de l'insertion d'une capsule quittant une station ou d'une capsule s'insérant dans une boucle après avoir été routée par un switch. Ce point devrait consister entre la simple incorporation du simulateur du second Projet Industriel des autres élèves de 3ème année à TELECOM Nancy travaillant sur un projet URBANLOOP. Le cas échéant et en attendant une solution plus réaliste et optimale il pourra s'agir d'un simple test de présence ou non avant de s'insérer dans la boucle.
- La partie visuelle de l'application étant actuellement une application web en local, il pourrait être intéressant de déployer notre projet sur un serveur et donc pouvoir y avoir accès sans avoir les sources à transmettre.
- Il pourrait être intéressant de tester différents algorithmes de routage, en particulier des gestions différentes de l'aspect de recalcule dynamique. Ce point fait d'ailleurs office de point de départ à un Projet Interdisciplinaire de Recherche ou de Développement (PIDR) d'élèves de deuxième année à TELECOM Nancy actuellement en cours. Ceci permettrait de vérifier par exploitation des performances que celui actuellement choisi est bien optimal.
- toujours d'un point de vue du routage le modèle actuel n'étant pas réaliste puisqu'une capsule vide boucle indéfiniment jusqu'à correction du défaut, il faudrait ajuster le fait que la capsule se dérouté vers la station accessible la plus proche de la destination voulue.
- De manière à optimiser davantage la répartition des capsules dans le réseau il pourrait être pertinent que les capsules vides puissent se dérouter vers les stations vides les plus proches en priorité plutôt que vers les entrepôts. Actuellement ce n'est le cas que si la station vide se trouve sur le chemin depuis la station qui a libéré la capsule, vers l'entrepôt qui récupère ladite capsule. Ceci permettrait aussi de savoir s'il est plus intéressant de voir les capsules vides boucler (et possiblement engorger à tort) dans le réseau ou si la solution des stockages dans les entrepôts est intéressante.
- la récupération des statistiques étant actuellement assez sommaire, une amélioration pourrait être d'exporter les données et éventuellement les exploiter sous forme de graphiques et ce, automatiquement. En outre il pourra être pertinent de rajouter des statistiques auxquels nous n'avons pour l'instant pas pensé.
- Le réseau de base étant simplifié puisque constitué uniquement de 3 boucles. Il peut être intéressant de travailler sur un réseau plus complet.

- URBANLOOP étant principalement composée de capsules individuelles, le simulateur ne considère que ces cas-là. En revanche le projet global prévoit la possibilité de capsules de transports particulières, comme par exemple celles pouvant accueillir deux voyageurs, celles avec emplacements pour les bagages ou encore pour personnes handicapées ou encore même des capsules ambulances (de type civières). Pour qu'un voyageur puisse utiliser ces capsules qui seraient en faible nombre il faudrait alors rajouter un système permettant d'appeler un de ces types de capsules et la capsule correspondante la plus proche démarrerait un trajet en direction de la station d'appel.

Conclusion

Rétrospective sur le projet

Nous avons abouti à un simulateur qui répond aux attentes initiales du projet industrielle. Bien qu'il ne permette pas encore de pouvoir simuler l'ensemble des situations pouvant arriver dans la réalité, il reste fidèle et cohérent avec ce que pourrait être le réseau de transport urbain URBANLOOP. Quelques points restent à améliorer cependant : nous sommes conscients que certains critères ne sont pas totalement satisfaits : par exemple, il reste encore un peu de travail à effectuer pour obtenir des statistiques en temps réel sur le réseau, ou encore la mise en production sur un serveur web du simulateur.

De plus, bien que non spécifié au démarrage du projet industriel, nous avons très vite compris l'intérêt et surtout le besoin pour l'équipe d'URBANLOOP d'avoir un visuel décrivant les simulateurs, et avons estimé que la livraison d'une interface graphique était essentielle pour les acteurs du projet. Nous livrons en ce jour une application web et un simulateur qui pourraient potentiellement être intégrés dans le futur au wiki d'URBANLOOP.

Nous livrons également un code source documenté pour le mieux, dans lequel nous avons essayé d'ajouter au maximum de la couverture, accompagné d'un manuel d'utilisation clair pour une prise en main efficace et rapide du simulateur.

Apport personnel et avis

Grâce à ce projet, nous avons chacun acquis de nouvelles connaissances et avons développé nos compétences de cohésion, notre capacité à travailler en groupe, notre communication avec nos partenaires, notre gestion de projet ainsi que nos *programming skills*. L'utilisation d'un simulateur à événements discrets était une première pour nous tous, et nous avons dû nous former afin d'exploiter les fonctionnalités qui s'offraient à nous. Nous avons également travaillé sur les algorithmes de routage pendant la phase d'études en nous penchant sur les méthodes existantes pour acheminer les paquets IP, et pour optimiser les algorithmes correspondants, afin de les appliquer à notre cas d'étude. De plus, nous avons renforcé notre maîtrise du langage Python ainsi que de la programmation orientée objet.

Nous sommes extrêmement fiers d'avoir eu la chance de pouvoir travailler sur le projet URBANLOOP. Ce fût une opportunité unique et espérons l'avoir honorée. Le projet est ambitieux, vendeur, futuriste et surtout synonyme de progrès. Bien qu'à ses prémices, nous espérons avoir apporté notre pierre à l'édifice et que notre contribution sera utile à l'équipe industrielle. Cette dernière nous a beaucoup motivés en nous amenant à nous dépasser, à rajouter des fonctionnalités au simulateur car nous ressentions l'intérêt que porte l'équipe à notre projet.

En ce qui concerne l'organisation du projet, nous regrettons que la phase d'études, bien qu'elle soit essentielle, ait été aussi longue. Nous avons aujourd'hui l'impression que cette phase a été trop étalée dans le temps et que nous n'avons pas été autant efficace que nous aurions pu l'être si nous avions commencé le développement un mois plus tôt. Nous avons honnêtement perdu de l'intérêt dans le projet pendant le mois de décembre car nous n'avons pas l'impression d'être réellement efficaces, d'autant plus que la plupart de nos camarades étaient d'ores et déjà en phase de développement sur leurs projets respectifs.

Nos vœux pour la suite

Ce fût une belle aventure pour nous et probablement le plus ambitieux et concret des projets sur lesquels nous avons travaillés. Nous espérons de tout cœur que notre travail et nos efforts porteront leurs fruits et que l'équipe industrielle arrivera à exploiter convenablement nos livrables.

Nous avons également entendu parler de stages et de projets industriels de recherche qui seraient dans la continuité de nos efforts, et cela nous conforte dans notre fierté d'avoir participé à un projet aussi ambitieux et autant prometteur. Nous souhaitons sincèrement à l'ensemble des acteurs du projet la réussite qu'ils méritent et que le projet ira au bout de son objectif. Encore une fois, merci à l'équipe du projet URBANLOOP de nous avoir donné la chance de collaborer avec elle.

Annexes

1 ANNEXE : ÉTAT DE L'ART

1 Annexe : État de l'art

Lors de la première moitié de projet, l'axe de travail était orienté vers la réalisation de diverses recherches pouvant mener à des études plus approfondies pour mener à bien le développement d'un simulateur de réseau URBANLOOP. Dans cette optique nous nous sommes intéressés aux solutions existantes pour faire le choix le plus adapté possible.

L'étude de l'existant a été scindée en deux parties : d'une part le choix d'un simulateur à événements discrets, qui consistera en la base de notre implémentation de la simulation. D'autre par l'étude des algorithmes de routage existants, paramètre phare dans notre conception.

1.1 Simulation à événements discrets

Pour donner suite à la spécification du projet, il est apparu évident qu'il nous fallait choisir un simulateur à événements discrets puisque notre projet a pour objectif de simuler un réseau de transport en temps réel. Nous avons écarté l'éventualité d'utiliser un simulateur de réseau IP à cause de la rigidité de ces derniers. Si l'on s'en tient à la définition d'une simulation à événements discrets :

« La simulation à événements discrets est une technique utilisée dans le cadre de l'étude de la dynamique des systèmes. Elle consiste en une modélisation informatique où le changement de l'état d'un système, au cours du temps, est une suite d'événements discrets. Chaque événement arrive à un instant donné et modifie l'état du système. ¹ »

Nous pourrions grâce à ce type de simulation gérer et automatiser les multiples événements déclenchés par le réseau de capsules URBANLOOP au fil du temps, tels que l'arrivée d'une capsule à une station, l'arrêt de celle-ci, la descente de passagers, etc.

Après avoir envisagé d'utiliser des logiciels de simulation tels que *Anylogic* [?] ou *FlexSim* [?], nous avons conclu que, pour des questions de droits d'utilisation mais aussi et surtout d'adaptation de l'outil, il nous fallait plutôt choisir une bibliothèque de simulation à événements discrets. Une telle bibliothèque permettra d'avoir un accès à une palette de fonctionnalités prédéfinies et utiles pour l'implémentation future.

Nous avons premièrement effectué une présélection rapide des simulateurs existants en éliminant les logiciels complets de simulation, ou les bibliothèques de simulation codées dans des langages qu'aucun membre du groupe ne maîtrise. Cette présélection a fait apparaître quatre bibliothèques de simulation à événements discrets intéressantes se prêtant convenablement au cadre de notre projet : Desmo-J (Java), SimJS (Javascript), SimPy (Python) et SystemC (C++).

Pour effectuer un choix entre ces 4 possibilités, nous avons établi une liste de critères décisifs pour déterminer la solution la plus avantageuse.

1. Source : Wikipédia

1.1.1 Critères d'évaluation

Comme énoncé précédemment, nous avons établi un tableau de critères nous permettant d'obtenir une note pour chacune des quatre bibliothèques de simulation. Ils se distinguent en deux catégories : des critères fonctionnels qui décrivent nos attentes concernant des fonctionnalités de base, et des critères non fonctionnels qui décrivent la qualité de l'environnement autour du simulateur (communauté, présence de tutoriaux pertinents, etc).

Critères non fonctionnels			Critères fonctionnels		
Tutoriels / exemples	/2		Nombre d'objets simulés suffisant (nombre de capsules/stations)	/3	
Communauté active / Forums	/2		Possibilité d'utiliser une interface graphique interne ou externe	/3	
Documentation	/2		Les objets et fonctions prédéfinis sont adaptés à notre projet (création d'un	/3	
Langage de programmation connu	/2		Possibilité d'injecter un jeu de données dans le simulateur et de l'utiliser	/3	
Projets similaires déjà réalisés	/1				
Licence libre	/1				
Sous-total	/10	0	Sous-total	/12	0

FIGURE 1 – Grille des critères d'évaluation des bibliothèques de simulation à événements discrets

Chacun de ces 10 critères est pertinent pour le choix du simulateur. Pour bien comprendre leur importance, en voici une description plus détaillée :

1. Présence de tutoriaux, d'exemples de code : pour pouvoir s'imprégner rapidement de la philosophie de rédaction et structuration du code dans une bibliothèque, il est très utile d'avoir à disposition des exemples pour pouvoir développer nos propres algorithmes.
2. Existence d'une communauté active : si des problèmes concernant l'utilisation d'une bibliothèque apparaissent en cours de projet, une demande d'aide sur des forums actifs peut débloquer la situation.
3. Documentation disponible : ce critère est primordial car une bonne utilisation des capacités d'une bibliothèque passe par une étude de sa documentation.
4. Langage de programmation maîtrisé par le groupe : ce critère est subjectif au groupe mais toutefois primordial pour un démarrage immédiat de l'implémentation sans apprentissage de langage au préalable.
5. Existence de projets similaires : des projets similaires peuvent nous permettre de concevoir l'architecture de notre projet de manière plus précise et sans faux pas.
6. bibliothèque *open source* : le code développé au cours de ce projet industriel sera également *open source*, nous devons donc absolument utiliser une bibliothèque libre de droit.
7. Nombre d'objets simulés simultanément : le langage et le simulateur utilisés doivent pouvoir supporter un nombre important d'objets à simuler simultanément.
8. Couplage du simulateur avec une interface graphique : une de nos tâches est de construire une visualisation de la simulation du réseau. Pour cela, le langage et le simulateur choisis doivent permettre la création d'une interface graphique décrivant les simulations.
9. Fonctions prédéfinies utiles : ce critère n'est pas primordial mais l'existence de fonctions utiles dans la bibliothèque de simulation permettrait un gain de temps non négligeable.
10. Injection et utilisation de jeux de données : l'injection de jeux de données permettrait de simplifier l'utilisation de statistiques récoltées en amont. Cela pourrait également servir pour une éventuelle sauvegarde de l'état du simulateur à un instant donné.

1.1.2 DESMO-J

DESMO-J est un acronyme pour Modélisation de Simulation d'Événements Discrets en Java [?]. DESMO-J apporte une palette de classes utiles pour la distribution, les queues, l'accomplissement et la planification de tâches, etc. Grâce à son infrastructure, l'utilisateur peut se concentrer uniquement sur son modèle à simuler en termes d'événements et de processus.

Ce simulateur obtient la note de **10/20** pour les raisons suivantes :

- [-] Il souffre d'une documentation quasi inexistante.
- [-] Peu de tutoriaux pour aider à la prise en main.
- [-] La communauté n'est plus active.
- [+] Le langage Java est très bien maîtrisé par le groupe.
- [+] De nombreux objets peuvent être simulés sans latence.

1.1.3 SimJS

SimJS est une bibliothèque JavaScript de simulation d'événements discrets[?]. La bibliothèque a été écrite afin de permettre la simulation dans les navigateurs standards. SimJS prend en charge les entités, les ressources, la communication (via des temporisations, des événements et des messages) et des statistiques (avec des séries de données, des séries temporelles et des statistiques de population). La distribution SimJS contient des didacticiels, une documentation détaillée et un grand nombre d'exemples.

Ce simulateur obtient la note de **9/20** pour les raisons suivantes :

- [-] La documentation existe mais est très peu claire.
- [+] Des exemples de code sont disponibles, bien que succincts.
- [-] Le langage Javascript n'est pas le point fort du groupe.
- [-] Le nombre d'objets pouvant être simulés est limité.

1.1.4 SimPy

SimPy est un *framework* de simulation d'événements discrets pour Python [?]. Son répartiteur d'événements est basé sur les générateurs de Python et peut également être utilisé pour la mise en réseau asynchrone ou pour la mise en œuvre de systèmes multi-agents (avec communication à la fois simulée et réelle). Les processus de SimPy sont de simples fonctions et sont utilisés pour modéliser des composants actifs tels que des clients, des véhicules ou des agents. SimPy fournit également divers types de ressources partagées pour modéliser des points de congestion à capacité limitée. La distribution SimPy contient des didacticiels, une documentation détaillée et un grand nombre d'exemples.

Ce simulateur obtient la note de **14/20** pour les raisons suivantes :

- [+] La documentation est complète.
- [+] De nombreux exemples de code permettent de débiter l'implémentation facilement.
- [-] La communauté environnante n'est pas très active.
- [+] Le langage Python est très facile à prendre en main.
- [+] De nombreux objets peuvent être simulés sans latence.

1.1.5 SystemC

SystemC est un langage de description permettant une modélisation de systèmes au niveau comportemental [?]. SystemC n'est pas un langage à part entière mais un ensemble de classes C++ qui introduisent les concepts nécessaires à la modélisation du matériel (par exemple la notion de processus concurrents). Conservant les fonctionnalités du C++, il reste possible de décrire des fonctions purement logicielles. SystemC intègre également la possibilité de simuler le modèle conçu, puis, par raffinements successifs, d'aboutir à une représentation implémentable.

Ce simulateur obtient la note de **12/20** pour les raisons suivantes :

- [+] La documentation est complète.
- [-] Le simulateur semble très voire trop complet, ce qui rend la compréhension difficile.
- [+] La communauté environnante est active.
- [-] Le langage C++ n'est pas maîtrisé par l'ensemble du groupe.
- [-] Peu d'objets peuvent être simulés sans latence.

1.1.6 Simulateur choisi

Le simulateur qui a obtenu la meilleure note est **SimPy**. Nous nous en sommes donc tenu à la notation et c'est celui-ci qui sera utilisé au cours de notre projet comme base de l'implémentation. Nous devons donc nous former à l'utilisation du langage Python durant les vacances de Noël. De plus, nous utiliserons les premiers jours de la semaine bloquée pour le projet industriel pour prendre en main ce simulateur.

En figure 2 vous retrouverez la notation en détail de tous les simulateurs :

Simulateurs		Desmo-J	SimJS	SimPy	SystemC
Critères					
Critères non fonctionnels					
Tutoriels / exemples	/2	0	1	2	1
Communauté active / Forums	/2	0	0	1	1,5
Documentation	/2	0	1	2	2
Langage de programmation connu	/2	2	1	1	0,5
Projets similaires déjà réalisés	/1	0	0	1	0
Licence libre	/1	1	1	1	1
Sous-total	/10	3	4	8	6
Critères fonctionnels					
Nombre d'objets simulés suffisant	/3	3	1	2	1
Possibilité d'utiliser une interface graphique	/3	2	3	3	3
Objets et fonctions prédéfinis adaptés	/3	2	2	1	2
Possibilité d'injecter un jeu de données	/3	1	0	1	1,5
Sous-total	/12	8	6	7	7,5
Total	/20	10	9	14	12

FIGURE 2 – Évaluation des bibliothèques suivant nos critères

1.2 Algorithme de Routage

L'une des parties centrales du projet comprend le choix du chemin que les capsules doivent emprunter pour se rendre d'un point à un autre. Pour déterminer l'algorithme de routage le plus adapté à la situation, il fallait d'abord se renseigner sur les différentes solutions existantes pour pouvoir avoir pleine connaissance des possibilités de choix et donc s'arrêter sur la solution la plus efficace dans notre cadre.

1.2.1 Routages classiques appliqués aux assistants de navigation GPS

Étant donné que le choix d'une route est un problème commun à tous les transports existants, il paraissait évident de s'intéresser aux algorithmes des assistants de navigation GPS existants pour les moyens de transports classiques. En effet, bien que notre réseau soit orienté et moins complexe qu'un réseau routier ou l'espace maritime, il est intéressant de comprendre comment, étant connues la position et la destination voulue, les assistants de navigation déterminent le plus court chemin en tenant compte du réseau (obstacles, orientation des routes...).

Dijkstra

L'algorithme de Dijkstra a été inventé dans le cadre de la recherche de trouver efficacement le plus court chemin [?]. Il s'agit d'un problème algorithmique célèbre dont le but étant, dans un graphe donné dont les arêtes sont munies de poids et étant donnée en entrée deux Sommets "départ" et "arrivé", de trouver un chemin de poids minimum entre ces deux sommets. Généralement, on applique l'algorithme de Dijkstra pour déterminer la liste de tous les plus courts chemins vers tous les autres nœuds du graphe. Cet algorithme est connu pour sa complexité moindre malgré le problème complexe puisqu'il effectue le calcul d'un chemin en un nombre de calculs proportionnel à $(m + n) \cdot \log(n)$ avec n le nombre de nœuds et m le nombre de routes. Le temps de calcul augmente donc suivant la taille du réseau mais reste raisonnable ce qui permet son application dans de nombreux domaines. Il est important de noter qu'il existe de très nombreuses variantes de l'algorithme de Dijkstra, bien que le principe reste le même.

Le concept de l'algorithme est d'effectuer un parcours en largeur : on augmente graduellement le rayon de recherche jusqu'à avoir découvert tous les nœuds et chemins optimaux y menant. Concrètement, on construit au fur et à mesure un sous-graphe où l'on classe les sommets par ordre croissant en fonction de la distance (somme des distances de chaque saut (arc empruntés)) à partir du sommet de départ. Initialement, on considère toutes les distances comme infinies, puis on affine en considérant le sommet de distance minimale du sous-graphe que l'on ajoute alors au sous-graphe en mettant à jour les distances entre les autres nœuds et le graphe et on itère ainsi de suite jusqu'à recouvrement optimal du graphe initial (ou arrivée à destination).

De manière plus détaillée, on peut considérer l'algorithme en pseudo code sur le *listing 1*.

Au départ (initialisation), on ajoute au sous-graphe, modélisé par P, uniquement le nœud A de départ en considérant sa distance avec lui-même comme nulle. On ajoute alors les voisins du nœud A à T.

Ensuite, on boucle jusqu'à ce que T soit vide et donc qu'il reste des nœuds atteignables à parcourir. À chaque itération, on ajoute au sous-graphe P les éléments de T pour le nœud le plus proche de P (donc avec un coût minimal) puis on regarde les voisins de ce nœud S qu'on vient de retirer de T et on met à jour les coûts de T en ajoutant à ce dernier les voisins et leur coût associé (somme entre ce qui nous permet d'aller au sommet S et la distance entre ce dernier

et son voisin) s'ils n'y sont pas déjà ou en ne gardant que le minimum entre celui existant et celui qu'on calcule.

```

1  'variables :'
2  - En entree : Le graphe (eventuellement oriente) et pondere
3      Le sommet de depart A
4  - Creation de deux listes de Sommets :
5      P, tous les noeuds dont on a defini le plus court chemin : <
6      destination, cout>
7      T, tous les noeuds accessibles a l etape en cours <destination,
8      cout_temporaire>
9  'initialisation :'
10 P contient le noeud de edpart <A, 0>, #prochain :-, dernier :-
11 T contient les voisins de A avec comme cout la distance entre A et V = d(
12 A,V). #prochain :V, dernier :A
13 'deroulement :'
14 Tant que il reste des noeuds à parcourir (dans T) :
15 on prend le noeud S de T ayant le plus faible cout C et on l enleve de T
16 en l ajoutant a P <S,cout_s> #prochain :N, dernier :N
17 on calcule les distances entre ce n\oeud S et ses voisins d(s,x)
18 si X n appartient pas à P
19 si X n appartient pas à T :
20 on ajoute <X, d(S,X)+cout_s > # prochain :N, dernier :S
21 sinon : on regarde le cout cout_x éassoci
22 on calcule new_c_x = d(S,X)+cout_s
23 si new_c_x < cout_x
24 on remplace dans T < x, new_c> #prochain :N, dernier :X

```

Listing 1 – Une variante de l'algorithme de Dijkstra en pseudo-code

Cet algorithme n'est bien entendu qu'une variante, en particulier dans une grande partie des cas on sauvegarde dans T puis P le prochain nœud (et donc le voisin de A qui permet le coût minimal) pour ce faire on sauvegarde $\langle V, d(A, V), V \rangle$ à l'initialisation puis on copie toujours le "prochain nœud" nous ayant permis d'atteindre S dans T, en particulier s'il y a une mise à jour (voir code "prochain"). On peut aussi sauvegarder le dernier nœud avant l'arrivée, pour ça il faut sauvegarder le nœud au moment de la mise à jour (voir code "dernier"). Enfin, on peut aussi sauvegarder le chemin entier sous forme d'une liste de nœuds. Dans certains cas, on peut ne pas avoir T mais chercher le nouveau nœud d'intérêt parmi tous les nœuds restants, ce qui fait donc plus de tests mais la mémoire est moins sollicitée.

L'algorithme de Dijkstra est très souvent utilisé car il a été prouvé et possède une complexité très faible ce qui lui permet d'être applicable sur un réseau conséquent. À ce titre, il existe de nombreuses variantes plus ou moins complexes qui permettent de l'adapter aux besoins mais aussi éventuellement de l'améliorer via des astuces heuristiques comme c'est le cas de l'algorithme A*.

A-STAR : algorithmes heuristiques

Un autre algorithme très célèbre : A* (*A-étoile* ou *A-star*) [?]. Il s'agit d'un algorithme de recherche de chemin proche de l'optimum basé sur Dijkstra mais en utilisant les données heuristiques pour réduire le nombre de calculs. Il est intéressant de noter que dans un cas d'une carte "normale" ce type d'algorithme trouve une solution quasi-optimale plus rapidement mais qu'il existe des effets de bords pouvant amener à des cas où la recherche est plus lente et/ou le résultat très éloigné du "meilleur" chemin.

Cet algorithme est pertinent dans le cas où l'on souhaite privilégier la rapidité de calcul plutôt que l'exactitude des résultats : on garde en effet comme "meilleur chemin" la première solution qu'on trouve ayant un coût correct.

Le concept de cet algorithme est d'effectuer une évaluation heuristique sur les nœuds pour avoir une idée du meilleur chemin qui y passe, puis visiter chacun de ces nœuds par ordonnancement de cette évaluation. Ainsi, en considérant qu'on a une idée de la direction globale à prendre pour se rendre à la destination, on ne favorise pas le nœud le plus proche à explorer comme c'est le cas avec Dijkstra mais le nœud étant dans la direction considérée comme celle de la destination. L'idée est d'essayer de se rapprocher du meilleur itinéraire en considérant celui "à vol d'oiseau" puis en contournant les obstacles. L'important est la manière dont on définit l'heuristique en effet si on définit comme "le plus court chemin par rapport à là où je suis" et donc un heuristique égal au coût on se retrouve dans le cas d'application de Dijkstra, si on est sur "le plus faible écart par rapport au trajet 'à vol d'oiseau'" on est cette fois-ci dans le cas présenté ci-dessus et celui usuel.

Concrètement, l'algorithme fonctionne par découverte des nœuds adjacents en donnant priorité à ceux qui sont les "plus proches" de la destination (les moins éloignés de la fonction heuristique choisie). Initialement, on estime une distance séparant le nœud de départ de celui d'arrivée (la distance à vol d'oiseau), on aura ensuite le coût heuristique d'un nœud qui correspondra à la distance parcourue entre ce dernier et le nœud de départ sommé à l'évaluation initiale (et donc la différence avec). L'algorithme maintient alors deux listes : celle des nœuds vérifiés (communément *closed list*) et celle des nœuds à vérifier au tour prochain (communément *opened list*). L'algorithme s'arrête lorsque la file d'attente des nœuds à tester (*opened list*) est vide : c'est qu'il n'y a aucun chemin possible entre la source et la destination. On retire successivement le nœud de la file d'attente ayant la plus faible heuristique, si c'est le nœud d'arrivée alors on reconstitue le chemin en utilisant la liste "fermée" pour y parvenir. Si ce n'est pas le nœud d'arrivée alors on considère tous les nœuds contigus des nœuds de la liste fermée comme atteignables et on calcule les coûts associés (somme du coût de l'ancêtre et de la distance entre les deux) que l'on sauvegarde avec le nœud dans la file d'attente en y ajoutant l'heuristique calculé (la distance 'à vol d'oiseau' entre ce nœud et l'arrivée) sauf si ces nœuds sont déjà dans la liste fermée.

1.2.2 Routage des paquets dans l'Internet

Le concept du choix d'une route d'un paquet IP au sein d'Internet est très complet et peut être transposable assez aisément dans notre situation. En effet, un paquet IP provient d'une machine source et va vers une machine de destination dont la position n'est pas forcément connue. De plus, le parcours du paquet est parsemé de passages par des routeurs qui ont pour vocation de sélectionner une route de sortie en fonction de l'adresse de destination et de la table de routage qu'il possède. En outre, la force de la table de routage de ces routeurs est qu'elle peut être configurée de différentes manières ce qui donne lieu à des algorithmes différents que nous avons étudié en essayant de les transposer dans le cadre de notre application. Il s'agit donc de vulgariser les processus mis en place et d'essayer de les adapter à notre projet de routage.

On peut d'ores et déjà considérer que les routeurs correspondent à nos embranchements et donc au choix d'aiguiller ou non la capsule sur l'autre boucle, les capsules sont assimilées aux paquets IP et les machines sources et destinations aux stations sources et destinations. Cependant, il y a de nombreuses adaptations à faire, ce que nous spécifierons plus en détail dans une partie dédiée à la solution choisie.

Routage Statique

L'une des manières de router les paquets IP au sein d'un réseau est d'effectuer un routage statique, c'est à dire une configuration manuelle. Le concept est assez intuitif : on associe directement à une adresse de destination un chemin à emprunter.

Router statiquement offre un avantage considérable : la connaissance du chemin emprunté entre deux machines et ce quel que soit l'état du réseau puisque ce chemin est configuré à l'avance et non susceptible de changer. Il permet aussi une économie la bande passante (car il n'y a aucun message pour le routage qui transite sur le réseau) et il est sécurisé puisqu'on ne diffuse pas d'informations sur le réseau susceptibles d'être écoutées.

Ce routage statique a cependant de fortes contraintes puisqu'il impose de faire la mise à jour des routes manuellement et ce sur tous les équipements du réseau, un à un. Ainsi si un équipement est ajouté, déplacé, en panne ou enlevé il est nécessaire de se connecter physiquement à chaque équipement dans le réseau pour changer la configuration. Ce fonctionnement impose donc des contraintes impossibles à tenir en milieu industriel et est donc très rarement utilisé. Le point intéressant à retenir reste néanmoins que dans un réseau il est recommandé d'utiliser ce routage statique pour les stations d'envoi et de réception : on associe un sous-réseau d'envoi au routeur qui se chargera de retransmettre l'information. De même, si la machine d'envoi fait partie du même sous-réseau que celle de destination, la machine envoie directement au destinataire sans transiter inutilement par un routeur. Néanmoins, la définition de la table de routage est toujours définie statiquement.

Routage Dynamique

Le routage dynamique a le fort potentiel de pouvoir être automatique et mis à jour, il est donc adaptatif : les tables de routages des routeurs changent suivant l'état du réseau.

Le principe de ce routage est que les routeurs s'échangent des informations concernant les commutations via des protocoles de routages : il y a donc diffusion périodique de messages qui alimentent et permettent d'appliquer des algorithmes effectuant la mise à jour automatique des tables de routage.

Ce routage est continu : tant qu'il y a des informations à échanger, les routeurs envoient des messages (que ce soit pour dire qu'il y a eu un changement, ou pour attester que le chemin est toujours valide). Il est aussi asynchrone, chaque routeur faisant la mise à jour par rapport aux données qu'il a reçu à l'instant précédent.

Ainsi, bien que le déploiement soit parfois complexe et que la diffusion de message peut créer une brèche de sécurité et une occupation de la bande passante, ce routage est généralement favorisé car beaucoup plus performant et flexible aux changements de topologie du réseau.

Il existe deux types de routage dynamique : Vecteur de Distance ou État de liaison, auxquels peuvent s'ajouter plusieurs améliorations possibles.

Vecteur de Distance Le concept de ce premier type de routage dynamique est que chaque nœud (routeur) annonce à ses voisins sa distance par rapport à tous les autres nœuds du réseau. Le protocole le plus connu se basant sur ce concept est le RIP (*Routing Information Protocol*).

Il s'agit d'un protocole distribué : aucun nœud n'a la vision complète du réseau. On ne sait pas si les voisins nous disant atteindre un nœud peuvent l'atteindre en une ou plusieurs étapes (nombre de sauts) ou même si les chemins des uns l'incluent ou non. Il ne connaît que sa propre table de routage qu'il construit en fonction des données qu'il reçoit. Chaque nœud a donc dans sa table tous les nœuds accessibles sous la forme <nœud, distance, prochain_nœud> (le

prochain nœud étant celui lui ayant indiqué que la destination était accessible en un minimum de distance (en considérant aussi la distance qui le sépare de ce dit annonceur).

Ce type de protocole s'appuie sur un calcul de la route par itération successive mettant à jour la table de routage suivant les informations. S'il y a une mauvaise gestion des informations il peut rapidement arriver des boucle où le paquet transite sans jamais parvenir à sa destination : le routeur ne connaissant pas le chemin qu'il initie il est possible qu'il revienne vers lui à un moment et il le transitera à la même passerelle s'il n'a pas été mis à jour.

État de Liaison Les protocoles basés sur le concept d'état de liaison se contentent d'informer uniquement sur les distances à ses voisins adjacents et ceux sur l'ensemble du réseau. En pratique, un nœud communique à tous les autres nœuds du réseau sa distance par rapport à ses voisins accessibles en un saut. Connaissant ainsi ces informations, le routeur peut construire sa propre table de routage. Le protocole le plus connu se basant sur ce concept est le OSPF (*Open Shortest Path First*).

Le principal point fort de ce protocole est la distribution de la topologie du réseau (et du coup de chaque liaison) sur tous les routeurs ainsi chacun des nœuds a une copie complète de la carte du réseau.

Les protocoles d'état de liaison se basent sur deux mécanismes : le fait que la dissémination de l'information soit fiable (*reliable flooding*) pour permettre la mise à jour des données propres à chaque routeur et le calcul des routes localement en utilisant les informations fournies.

Pour disséminer les informations, chaque routeur met les éléments décrivant les liaisons dans des paquets appelés LSP (Link State Packet) ou LSA (Link State Advertisement) contenant l'adresse du nœud l'ayant créé, la liste des voisins directement connectés et le coût pour les joindre, un numéro de séquence et une durée de vie (*time-to-live*) attestant l'âge du paquet. À ce mécanisme s'ajoute aussi la détection de potentielle panne d'un routeur via un protocole "hello" qui consiste à se signaler régulièrement au routeur désigné pour qu'il puisse attester que tous les routeurs sont bien fonctionnels.

Le calcul de la table de routage est un algorithme de Dijkstra explicité ci-dessus à ceci près que l'on ajoute l'élément "prochain nœud" et lorsqu'on actualise le coût pour aller à un nœud, le "prochain nœud" reste toujours un voisin du nœud de départ ce qui permet d'avoir une table de routage directement lisible.

Ces mécanismes entraînent deux inconvénients à considérer :

- L'inondation (*flooding*), qui consiste à tenir informer de la situation locale tous les autres nœuds du réseau peut rapidement saturer la bande passante. En effet, non seulement il faut envoyer les informations périodiquement pour informer le réseau que les liaisons sont bien en état, mais il s'agit de diffuser l'information à tous les routeurs.
- Le calcul des meilleures routes est effectué localement en utilisant toute la carte à chaque calcul ce qui implique une grande utilisation mémoire mais aussi beaucoup de calculs. En effet, même si le coût de calcul de Dijkstra est optimisé, la mise à jour implique de refaire encore une fois l'algorithme et ce dans chaque routeur ce qui peut rapidement augmenter le nombre de boucles de calculs à faire.

Améliorations existantes Pour améliorer les algorithmes de routage dynamique et en particulier l'exploration du meilleur chemin possible, sans que cette recherche itérative entraîne un bouclage ou des diffusions de fausses informations venant biaiser le choix du chemin, il existe plusieurs solutions :

- Représenter la "valeur infinie" par une valeur finie. Cette technique permet d'éviter le comptage à l'infini (*count-to-infinity*) qui se produit lorsque la coupure d'une liaison en-

traîne une mise à jour en surenchères : les voisins pensent donc que la situation s'améliore mais elle finit en réalité par se rétrograder indéfiniment. En majorant la distance d'accès par cette "valeur infinie" on peut déterminer quand une position est devenue non atteignable. Il faut cependant bien définir la valeur choisie car si elle est trop faible, il se peut qu'une réelle distance ne soit pas considérée et donc de considérer un nœud atteignable comme non atteignable.

- L'horizon partagé (*Split-horizon*). Il s'agit de ne pas annoncer la route possible à un voisin N si ce dernier est le prochain nœud de la route vers la destination. Le but est d'éviter que le nœud N, perdant sa connexion vers la destination, choisisse de passer par le nœud qui passait par N pour atteindre cette destination, et donc se retrouver dans la situation où un nœud A veut passer par B pour rejoindre C alors que pour rejoindre C, B envoie le message à A.
- Le retour empoisonné (*Poison-reverse*). Se place dans le cadre de l'horizon partagé : non seulement on ne renvoie pas au "prochain nœud" que l'on peut atteindre la destination, mais pour l'empêcher de repasser par nous on lui annonce une route infinie vers cette destination.
- Temporiser lorsque l'on reçoit une distance infinie (*hold-down*). Le but est là encore d'éviter de transmettre de mauvaises informations. Ainsi, lorsque l'on reçoit une distance infinie, avant que le routeur n'annonce qu'il peut atteindre la destination, il attend un certain laps de temps pour laisser à son "prochain nœud" de cette destination, le temps nécessaire pour revoir sa table connaissant l'annonce de l'infinie préalable et éventuellement considérer qu'il ne peut plus accéder à la destination et donc l'annoncer au routeur qui s'apprêtait à diffuser une fausse information.
- Récupérer le chemin émetteur (*Source tracing*). Il s'agit de garder, en plus du prochain nœud, la totalité le chemin parcouru. Ainsi, lorsqu'un routeur annonce une distance infinie, s'il est inclut dans le chemin, on actualise en considérant la source comme inaccessible et on recalcule, plutôt que d'annoncer à tort qu'on peut y accéder (en passant par le chemin qui n'est plus valide justement).
- Utiliser des procédés heuristiques qui permettent un calcul plus efficace de l'optimum. Il s'agit de pondérer la recherche pour ne pas effectuer les itérations qui s'éloignent trop du chemin optimal. Utiliser l'algorithme A^* en est un exemple.

Certains éléments du routage inter-domaines ont aussi été relevés, en particulier l'étude du Border Gateway Protocol (BGP) dans lequel la politique d'acheminement des paquets dans un réseau interne n'est pas le même que quand le paquet doit être acheminé en passant par l'extérieur. En effet, en interne, il s'agit d'un réseau de confiance et donc l'acheminement se fait de manière classique, mais il y a des routeurs qui ont un rôle de "contrôleur de frontière" filtrant les accès. Ces derniers en profitent aussi pour rendre invisible la topologie de l'espace interne : l'AS (*Autonomous System*) dont il fait part.

1.2.3 Choix du routage

Après avoir pris connaissance des différentes solutions actuellement existantes, nous avons choisi d'adapter un routage en sélectionnant des éléments de plusieurs solutions existantes : globalement il s'agit d'un routage statique avec une correction dynamique en cas d'anomalie. De plus, nous avons décidé de procéder par itérations et donc d'implémenter d'abord un algorithme fonctionnel basique, puis d'ajouter des éléments d'heuristique et des adaptations.

Annexe 2 : Manuel d'utilisation de l'application

Manuel d'utilisation du Simulateur URBANLOOP

1. Généralités :

1.1. Présentation

Cette application a pour objectif de modéliser le réseau URBANLOOP : réseau de transport urbain par capsules dans des tubes dont le réseau est constitué de boucles interconnectant.

L'application est constituée :

- D'un simulateur lequel modélise l'ensemble des éléments du réseau (les boucles, les stations, les entrepôts et les aiguillages ainsi que les voyageurs) et leurs interconnexions les uns avec les autres.
- D'une interface graphique permettant la visualisation des éléments générées et facilitant l'intervention des utilisateurs du réseau sur la configuration.

1.2. Prérequis

Pour lancer la simulation il faut avoir **Python3** avec les bibliothèques :

- | | | | |
|----------------------|----------|----------------|-----------|
| - SimPy ¹ | - flask | - configparser | - logging |
| - math | - random | - numpy | - scipy |
| - os | - time | - threading | - sys |
| - json | - copy | - fileinput | - shutil |
| - enum | - uuid | | |

Par ailleurs pour pouvoir avoir accès à l'interface graphique il faut une connexion internet puisque l'application importe dynamiquement les API nécessaires.

1.3. Où trouver ce que je cherche dans ce manuel ?

La partie 2. *Usage et compréhension des options* présente la manière d'utiliser les options qu'offre l'application mais aussi ce le comportement du simulateur et ce qu'il est possible de faire.

Dans les deux cas l'application est un système dynamique qui simule la "vie" du réseau URBANLOOP, néanmoins l'utilisateur peut interférer sur certains éléments de la configuration : changer le réseau et modifier les paramètres, ce que nous présentons dans la partie 3. *Configuration adaptative*.

Usage et compréhension des options	2
Lancement :	2
Lancer le simulateur uniquement	2
Lancer le simulateur avec l'interface graphique	3
Éléments du simulateurs :	3
Génération des voyageurs et choix de leur trajets	3
Routage	3
Répartition uniforme des capsules	3
Éléments de l'interface graphiques :	4
Représentation des éléments dans le modèle	4
Temps	5
Données de l'objet sélectionné	5
Configuration (cf partie 3.2)	5
Configuration adaptative	6
Topologie du réseau	6
Paramètres	7

¹ <https://simpy.readthedocs.io/en/latest/>

2. Usage et compréhension des options

2.1. Lancement :

Pour lancer l'application il y a deux possibilités :

Lancer le simulateur uniquement

Le simulateur est alors en temps réel et on peut travailler sur les logs de retours s'affichant en ligne de commandes. Pour se faire il faut compiler et lancer le fichier `/runnable/main.py`. Ceci peut être facilité par le raccourcis dans un terminal `./run main.py`

Il y a deux niveaux de verbosité des logs pertinents : INFO ou DEBUG. Cet élément peut être changé dans le fichier `/settings/simlog.py`, à la dernière ligne :

```
logging.basicConfig(format='[% (levelname)s]%(message)s', level=logging.DEBUG)
```

Dans cette configuration les éléments sont décrits état par état avec plus ou moins de détails.

En particulier la structure est de la forme :

[verboité][heure de l'événement][station ou boucle concernée (avec éventuellement un second si il s'agit d'un trajet)] Explications

Il y a différents événements générant des logs :

INFO :

- lorsqu'un voyageur arrive dans la file d'attente d'une station
- lorsqu'un voyageur monte dans une capsule
- lorsqu'une capsule part d'une station
- lorsqu'une capsule passe par un aiguillage : si elle reste sur sa boucle / change de boucle
- lorsqu'une capsule arrive dans une station
- lorsqu'un voyageur descend d'une capsule
- lorsqu'un défaut est généré
- si une station est pleine et donc ne peut pas accueillir une capsule dont la station était sa destination

DEBUG :

- Chaque passage d'une capsule par une station, un entrepôt ou un switch lors de son trajet
- Lorsque les processus de drainage/remplissage des stations sont lancés par les entrepôts
- Les changements d'états du simulateur (vitesse/pause)

```
[DEBUG][08:06:11][Switch n°2->Switch n°0] Capsule 39 continues its trip to Stanislas
[INFO][08:06:11][Loop:Nancy] Capsule 39 stays on its loop
[DEBUG][08:06:13][Switch n°0->Switch n°0] Capsule 9 continues its trip to TELECOM Nancy
[DEBUG][08:06:13][Switch n°3->Fac. de Lettres] Capsule 16 continues its trip to Fac. de Lettres
[INFO][08:06:13][Fac. de Lettres] Capsule 16 (1 traveler) ends its trip
[DEBUG][08:06:13][Fac. de Lettres] Contains 1 capsules now
[DEBUG][08:06:13] Start descent in capsule 16
[DEBUG][08:06:15][Switch n°2->Switch n°0] Capsule 31 continues its trip to Velodrome
[DEBUG][08:06:15][Switch n°2->Switch n°0] Capsule 33 continues its trip to Eglise
[DEBUG][08:06:15][Switch n°2->Switch n°0] Capsule 35 continues its trip to Eglise
[DEBUG][08:06:15][Warehouse Villers-les-Nancy->Switch n°1] Capsule 41 continues its trip to Nancy Thermal
[INFO][08:06:15][Loop:Villers-les-Nancy] Capsule 41 stays on its loop
[DEBUG][08:06:15][Warehouse Villers-les-Nancy->Switch n°1] Capsule 43 continues its trip to Nancy Thermal
[INFO][08:06:15][Loop:Villers-les-Nancy] Capsule 43 stays on its loop
[DEBUG][08:06:15][Warehouse Villers-les-Nancy->Switch n°1] Capsule 45 continues its trip to Stanislas
[INFO][08:06:15][Loop:Villers-les-Nancy] Capsule 45 stays on its loop
[DEBUG][08:06:17][Auchan->Sapiniere] Capsule 19 continues its trip to Sapiniere
[INFO][08:06:17][Sapiniere] Capsule 19 (Empty) ends its trip
[DEBUG][08:06:17][Sapiniere] Contains 1 capsules now
[DEBUG][08:06:17][Auchan->Sapiniere] Capsule 5 continues its trip to Sapiniere
[INFO][08:06:17][Sapiniere] Capsule 5 (Empty) ends its trip
[DEBUG][08:06:17][Sapiniere] Contains 2 capsules now
[INFO][08:06:17][Sapiniere->Gare] Traveler 30 gets in capsule 19
[INFO][08:06:17][Sapiniere->Velodrome] Traveler 42 gets in capsule 5
[DEBUG][08:06:18][Switch n°0->Velodrome] Capsule 9 continues its trip to TELECOM Nancy
[DEBUG][08:06:18][Switch n°2->Switch n°0] Capsule 25 continues its trip to Gare
[INFO][08:06:18][Loop:Nancy] Capsule 25 stays on its loop
[DEBUG][08:06:18][Auchan->Sapiniere] Capsule 20 continues its trip to Sapiniere
[INFO][08:06:18][Sapiniere] Capsule 20 (1 traveler) ends its trip
[DEBUG][08:06:18][Sapiniere] Contains 1 capsules now
[DEBUG][08:06:18] Start descent in capsule 20
[DEBUG][08:06:19][Switch n°0->Velodrome] Capsule 31 continues its trip to Velodrome
[INFO][08:06:19][Velodrome] Capsule 31 (Empty) ends its trip
[DEBUG][08:06:19][Velodrome] Contains 1 capsules now
[DEBUG][08:06:19][Switch n°0->Velodrome] Capsule 33 continues its trip to Eglise
[DEBUG][08:06:19][Switch n°0->Velodrome] Capsule 35 continues its trip to Eglise
[INFO][08:06:19][Velodrome->Gare] Traveler 29 gets in capsule 31
[DEBUG][08:06:21][Switch n°2->Switch n°0] Capsule 37 continues its trip to Gare
[INFO][08:06:21][Loop:Nancy] Capsule 37 stays on its loop
[INFO][08:06:22] Traveler 31 gets out capsule 16
[DEBUG][08:06:23][Switch n°3->Switch n°2] Capsule 17 continues its trip to Campus_ARTEM
[INFO][08:06:23][Sapiniere] Capsule 19 (1 traveler) starts its trip to Gare
[DEBUG][08:06:23][Sapiniere] Amount of capsules : 1
[INFO][08:06:23][Sapiniere] Capsule 5 (1 traveler) starts its trip to Velodrome
[DEBUG][08:06:23][Sapiniere] Amount of capsules : 1
```

Sortie en terminal des logs au niveau DEBUG

Lancer le simulateur avec l'interface graphique

de manière à avoir accès à la visualisation et une interface intuitive pour effectuer des configuration.

Pour se faire il faut compiler et lancer le fichier `/runnable/web_app.py`. Ceci peut être facilité par le raccourci dans un terminal `./run web_app.py`

Une fois fait il faut aller dans un navigateur de notre choix à l'adresse locale <http://127.0.0.1:8090/>

2.2. Éléments du simulateur :

Le simulateur a été implémenté de manière à simuler le réseau le plus réalistiquement possible sans l'intervention d'un utilisateur, mais pouvant aussi s'adapter aux interactions d'un utilisateur avec l'interface. De manière générale le simulateur effectue plusieurs éléments.

Génération des voyageurs et choix de leur trajets

Régulièrement et suivant des modèles statistiques, des voyageurs sont générés dans le réseau. Cette génération a été effectuée de manière à correspondre le plus possible à des cas de réalité, ainsi il y a une plus forte probabilité qu'un voyageur arrive dans le réseau aux heures de pointe (8h et 18h).

En outre un voyageur arrive dans une station particulière pour aller vers une autre station, les deux décidées par le simulateur. Là encore la modélisation se voulant réaliste, les apparitions sont dictées par un modèle statistique qui favorise en particulier les choix de trajets suivant l'heure, le matin des zones résidentielles aux zones d'activités, et l'inverse le soir.

Cette répartition d'apparition suivant les heures peut être ajustée dans le fichier `/simulator/traveler_generator.py`, de même le choix de la station d'apparition et de destination dans le fichier `/simulator/traveler_generator.py`.

Routage

Le choix de la route pour aller à une destination donnée est effectué automatiquement par le simulateur. En effet, le réseau des boucles, maillés par les aiguillages, permet un calcul du "plus court chemin" par un algorithme de Dijkstra. Chaque aiguillage sait donc, en fonction de la destination d'une capsule qui arrive à son niveau, s'il doit ou non aiguiller la capsule vers l'autre boucle.

Le coût d'un trajet qui est minimisé est calculé suivant la distance au sein d'une boucle et en considérant un coût plus élevé s'il faut traverser un aiguillage (ce coût supplémentaire est représenté graphiquement par l'interconnexion entre deux boucles, auquel peut être rajouter une constante configurable).

Le routage initial est statique mais lorsqu'il y a une anomalie que l'utilisateur ajoute, le "nouveau chemin optimal" est calculé en limitant les passages par la section entre deux aiguillages qui est concernés. Dans la réalité ceci correspondrait à un besoin de ne pas passer par une section de boucle pour maintenance ou si l'arc est saturé.

Répartition uniforme des capsules

Chaque station a un nombre limité de slots pouvant accueillir des capsules, ce nombre représente la capacité de la station. Comme dans un cas réel, il peut arriver qu'une station soit assaillie de voyageurs, ou encore que toutes les capsules se rendent à la même station à un temps donnée. Pour pallier cette possibilité le simulateur offre la possibilité de drainer les capsules vides qui encombrant une station contenant trop de capsules pour sa capacité et inversement de remplir les stations vides par des capsules pour que les voyageurs arrivants puissent monter dans une capsule.

Dans cette optique nous avons inclus dans le réseau les entrepôts, qui ne sont la destination d'une capsule que dans le cas où elle est vide. Ils permettent d'effectuer un stockage isolé des capsules pour limiter les transports à vide mais surtout optimiser l'uniformisation du nombre de capsules dans le réseau. Ainsi une station trop remplie qui n'a pas de voyageurs va libérer des capsules vides en direction de l'entrepôt le plus en aval. Inversement une station n'ayant que peu de capsules vides avec des voyageurs en attente de partir va se voir être la destination de capsules partant de l'entrepôt le plus en aval.

Par ailleurs, cet équilibrage du nombre de capsules s'effectue au regard des horaires : on remplira de capsules vides les zones résidentielles le matin, et les zones d'activités le soir, pour que les voyageurs arrivant en grand nombre à ces heures puissent attendre le moins possible, et on videra les stations de zones d'activités le matin et de zones résidentielles le soir pour que les voyageurs dans les capsules ayant pour destination ces stations, puissent avoir des emplacements de libre dans les stations.

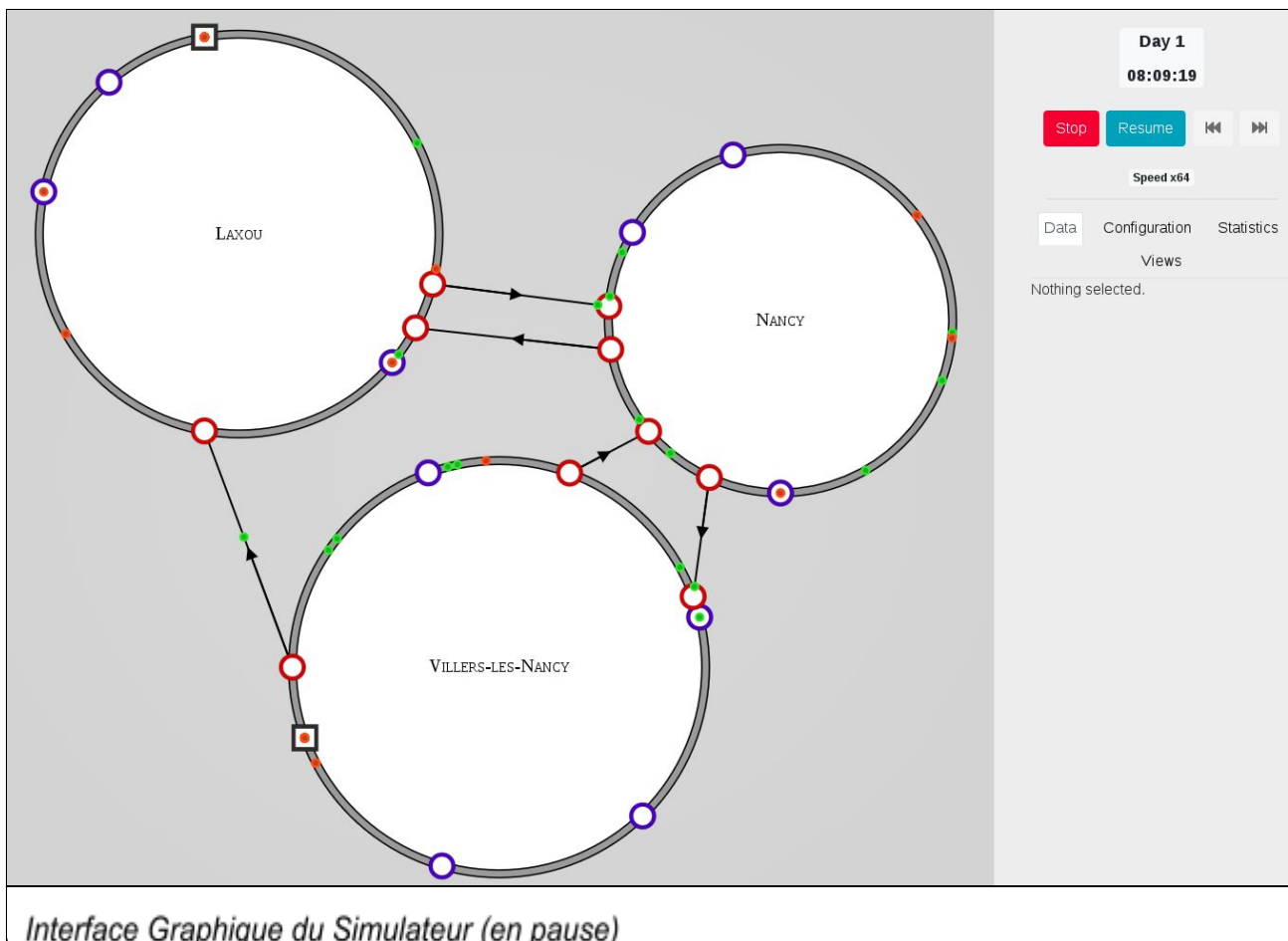
2.3. Éléments de l'interface graphiques :

L'interface graphique se présente sous la forme d'une fenêtre principale contenant le réseau modélisé et d'un panneau de réglage. La capture ci-dessous a été faite lorsque le simulateur était en pause.

Représentation des éléments dans le modèle

Dans la fenêtre contenant le réseau on trouve :

- Les boucles (loops) représentées par de larges cercles blancs cernés d'un "rail" gris. En leur centre, le nom des boucles est inscrit.
- Les stations sont les cercles bleus sur le périmètre des boucles
- Les entrepôts (warehouses) sont représentés par les carrés noirs, sur ces mêmes "rails"



Interface Graphique du Simulateur (en pause)

- Les aiguillages (switchs) correspondent à deux cercles rouges chacun sur une boucle différente interconnectés par une flèche qui indique le sens. La longueur du trait représente le coût de traversé.
- Les capsules sont les ronds verts et rouges circulants sur les rails des boucles, sur l'interconnexion d'un aiguillage ou bien stockées dans les stations ou les entrepôts. Les ronds sont verts si la capsule contient un voyageur, rouge sinon.

Il est possible de sélectionner un objet en cliquant dessus, il se cernera alors d'un liseré jaune.

Il est possible de zoomer ou dézoomer la fenêtre du réseau.

Une barre d'échelle des objets est aussi disponible en bas du panneau de réglage pour agrandir les objets autres que les boucles.

Temps

Le cadre le plus en hauteur dans le panel de réglage affiche le temps déroulé depuis le lancement de la simulation. L'heure de lancement peut d'ailleurs être changée.

Pour lancer la simulation il faut appuyer sur le bouton **Start** vert, qui devient le bouton **Stop** après appuie. Appuyer sur le bouton **Stop** réinitialise la simulation. Il est aussi possible, après lancement, de mettre en pause et relancer la simulation en appuyant sur le bouton turquoise alternativement **Pause** et **Resume**.

Au lancement la simulation est en temps réel. Les boutons d'**accélération** et de **décélération** permettent de moduler la vitesse du simulateur. La vitesse en question est affichée en dessus de ces boutons.

Données de l'objet sélectionné

Le panel d'information contient un onglet **Data** qui permet d'afficher le détail des éléments sur lesquels on clique. En particulier on peut avoir différentes informations :

- Pour les boucles : le nom, le sens de rotation (horaire ou non) mais aussi la liste des composants.
- Pour les stations : le nom, la boucle contenant, l'élément d'après, la capacité, mais aussi la taille de la file d'attente des voyageurs ou encore le nombre de capsules.
- Pour les entrepôts : la boucle contenant, l'élément suivant, la capacité et le nombre de capsules actuellement.
- Pour les aiguillages : les boucles desservies avec l'élément suivant dans chacune et la longueur du lien
- Pour les capsules : si elles contiennent ou non un voyageur, leur destination, leur position et leur future position ainsi que la boucle sur laquelle elles circulent.

-

Configuration (cf. partie 3.2)

Dans le panneau de réglage, l'onglet **Configuration** permet de changer les valeurs par défaut pour étudier leur impact. Les changements peuvent être appliqués temporairement ou de manière permanente (devenir les éléments par défaut). La section 3.2 présente plus en détails ces paramètres modulables.

3. Configuration adaptative

L'application, aussi bien avec que sans interface graphique, se veut adaptative. Il est donc possible de changer différents éléments.

3.1. Topologie du réseau

La topologie est totalement chargée à partir d'un fichier `json` devant se trouver dans `/resources/networks/`, elle peut donc facilement être modifiée. De plus l'interface graphique adaptera la taille du réseau visible en fonction du fichier chargé.

La structure du fichier est cependant importante. L'image ci-dessous en est un exemple où les éléments en violet sont les "clés" obligatoirement présentes.

```
{
  "Nancy": {
    "center": [3300,2600],
    "clockwise": false,
    "circonference": 5000,
    "content": [
      {"type": "station", "name": "Gare", "angle": 149.2, "station_type": 3, "capacity": 5},
      {"type": "station", "name": "Stanislas", "angle": 106, "station_type": 3, "capacity": 4},
      {"type": "station", "name": "Campus ARTEM", "angle": 270, "station_type": 1, "capacity": 4},
      {"type": "switch_out", "other_loop": "Villers-les-Nancy", "angle": 245.6},
      {"type": "switch_out", "other_loop": "Laxou", "angle": 189.6},
      {"type": "switch_in", "other_loop": "Laxou", "angle": 175.2},
      {"type": "switch_in", "other_loop": "Villers-les-Nancy", "angle": 220}
    ]
  },
  "Villers-les-Nancy": {
    "center": [2000,1000],
    "circonference": 6000,
    "clockwise": true,
    "content": [
      {"type": "switch_in", "other_loop": "Nancy", "angle": 20},
      {"type": "station", "name": "Velodrome", "angle": 14, "station_type": 3, "capacity": 5},
      {"type": "station", "name": "TELECOM Nancy", "angle": 314, "station_type": 1, "capacity": 4},
      {"type": "station", "name": "Eglise", "angle": 254, "station_type": 2, "capacity": 3},
      {"type": "warehouse", "capacity": 50, "angle": 200},
      {"type": "switch_out", "other_loop": "Laxou", "angle": 180},
    ]
  }
}
```

Extrait d'un JSON respectant la structure de réseau du simulateur URBANLOOP

La structure est donc :

```
{ "<Nom_de_la_boucle>": {
  "center": [<x>, <y>], correspond aux coordonnées du centre de la boucle
  "clockwise": <true/false>, sens horaire de rotation
  "circonference": <taille>, valeur en mètre
  "content": [ { "type": <station/switch_in/switch_out/warehouse>,
    "angle": <dans_cercle_trigonometrique_degre>, "name": <nom_station>,
    "station_type": <[0-4]> 0= neutre, 1=zone d'activité, 2=zone résidentielle, 3=centre
    ville>,
    "capacity": <nombre_max_de_capsules>,
    "other_loop": <nom_autre_boucle> },
    {...}, ...
  ]
},
"<Autre_boucle>" : {...
...
}
```


3.2. Paramètres

Plusieurs paramètres sont modulables dans le réseau.

Pour modifier les éléments il existe deux moyens :

- Le premier en modifiant directement le fichier `/resources/config.ini`
- Le second depuis l'onglet **Configuration** du panneau de réglage de l'interface graphique.

Dans le second cas il faut que la simulation soit arrêtée (pas lancée ou **Stop**).

Les différents paramètres configurables sont répartis en plusieurs catégories avec une indication sous chaque champs de paramètres. Les valeurs actuelles sont présentes au chargement de la page.

À noter qu'il est possible de mettre -1 pour qu'il n'y ait pas de limitation :

VOYAGERS (travelers)

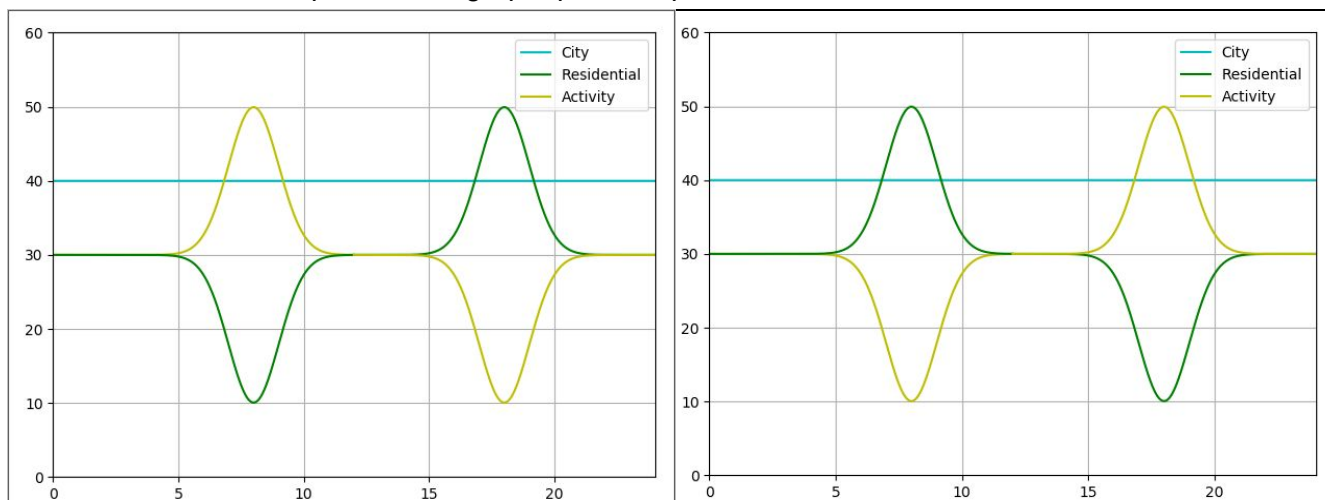
- le nombre de voyageurs par jour en moyenne
- le nombre maximum de trajets effectués par jour
- le nombre maximum de voyageurs par jour
- le temps moyen de descente et montée des voyageurs dans les capsules en secondes
- Le pic de l'heure de pointe du matin
- Le pic de l'heure de pointe du soir

MODÈLE PROBABILISTE (probabilities)

correspond au modèle des heures de pointe : à quel point la différence est forte. Le double du premier champ sommé avec le second doit faire 100.

- Le pourcentage de la probabilité d'apparition qui sera répartie durant les heures de pointe pour les zones résidentielles et celles d'activités (divisée par 2)
- Le pourcentage d'arrivée pour les zones de centre-ville (le restant)
- La fluctuation durant les heures de pointe, qui doit valoir moins que le premier champ.

Ces critères correspondent au graphique de répartition des arrivées et destination ci-dessous.



Graphiques des probabilités d'apparition et de destination des voyageurs selon les stations en fonction de l'heure.

Réglages : Probabilité d'apparition et d'être à destination du centre ville = 40%

Fluctuation en heure de pointe = 20

Probabilité d'apparaître et d'être à destination des zones d'activités = probabilité d'apparition et de destination en zones résidentielles = 30%

CAPSULES

- La vitesse (constante) des capsules dans le réseau, en mètres par secondes.
- Le nombre de capsules totales dans le réseau. S'il est illimité (-1) chaque station et entrepôt est rempli à sa capacité -1. Sinon elles sont réparties uniformément d'abord pour remplir les stations à capacité - 1, puis dans les entrepôts.

ROUTAGE (routing)

- le coût de traversé d'un aiguillage (ajouté à la longueur de la topologie)
- Le temps entre chaque envoi de notifications aux autres aiguillages que l'aiguillage est toujours fonctionnel (en secondes)
- Le temps avant de vérifier que les autres aiguillages sont toujours opérationnels (en secondes, supérieur au premier timer)

TOPOLOGIE (topologie)

- le fichier `json` (au format décrit dans le paragraphe 3.1) de la topologie du réseau

SIMULATION

- forcer ou non le temps réel
- imposer une durée ou bien le faire "sans fin"
- Choix de l'heure de lancement de la simulation.

Contacts en cas de questions :

- charlotte.asselin-boulle@telecomnancy.eu
- baptiste.chocot@telecomnancy.eu
- thibault.henry@telecomnancy.eu
- (encadrant) thibault.cholez@telecomnancy.eu

Références

- [1] Site officiel du projet URBANLOOP, <http://urbanloop.fr/> 2
- [2] Site de la métropole du Grand-Nancy, section transport en commun, <https://www.grandnancy.eu/se-deplacer/en-transports-en-commun/> 3
- [3] Site du réseau Stan, réseau de transports en commun de Nancy
http://www.reseau-stan.com/presentation/?rub_code=12 3
- [4] Page Wikipédia sur les Simulateurs à Événements Discrets, contient la liste sur laquelle nous nous sommes basés pour les recherches https://en.wikipedia.org/wiki/List_of_discrete_event_simulation_software
- [5] Site de la librairie DESMO-J, <https://en.wikipedia.org/wiki/DESMO-J> 5
- [6] Site de la librairie SystemC, <https://fr.wikipedia.org/wiki/SystemC> 5
- [7] Documentation officielle de la librairie SimPy, <https://simpy.readthedocs.io/en/latest/> 5
- [8] Site officiel SimPy, bibliothèque Python de simulation à événements discrets
<https://simpy.readthedocs.io/en/latest/index.html> 5
- [9] Site de la librairie SimJS, <http://simjs.com/> 5

Algorithme de Routage en pseudocode typé python

```
1 '''Un aiguillage (de A-->B) possede :
2   - moi : mon numero d'aiguillage
3   - un troncon avant moi : A-avant
4   - une boucle_presente : A-apres
5   - une boucle_aiguillee : B
6   - une longueur : ma_longueur
7   - une table de routage
8   - une table permanente : "quand tout va bien"
9   - une liste de timers correspondant au delai sans nouvelle
10  - un tableau de booleens d'anomalies (0 : sur boucle presente , 1 sur
    boucle aiguillee)
11 ,,,
12
13 #####INITIALISATION : calcul de la table pour la premiere fois
14 #table de routage permanente : {Boucle_destination:[doit_router , distance ,
    chemin]}
15 Table_Permanente = { presente:[ False , 0, [moi,presente]],
16                      aiguillee:[ True, ma_longueur, [moi,aiguillee ]]}
17 #table contenant les boucle accessible a tester
18 a_parcourir = {presente:moi, aiguillee:moi} #boucle_a_tester:
    dernier_aiguillage
19 Table_Permanente = parcours(Table_Permanente , a_parcourir)
20 GLOBAL Table = Table_Permanente
21
22 def parcours(Table , a_parcourir):
23     table_tempo = Table
24     a_parcourir = a_parcourir
25     while ( a_parcourir not vide ):
26         boucle_choisie = {min(boucle.distance) with (boucle in a_parcourir)}
27         aiguillage_etape = a_parcourir.enleve(boucle_choisie)
28         etape = table_tempo.get(boucle_choisie)
29         for all (aiguillage != aiguillage_etape) in boucle_choisie :
30             if (!aiguillage.anomalies[0]): #il n'y a pas d'anomalie pour rester
    sur la boucle
31                 table_tempo.add( {aiguillage.boucle_suite:
32                                     [etape.doit_router , etape.distance+d(aiguillage , aiguillage_etape
    ) , etape.chemin+[aiguillage , boucle_suite] ]
33                                     })
34                 a_parcourir.add(boucle_suite:aiguillage)
35                 if (!aiguillage.anomalies[1]): #il n'y a pas d'anomalie pour changer
    de boucle
36                     table_tempo.add( {aiguillage.boucle_aiguillee:
37                                         [etape.doit_router , etape.distance+d(aiguillage , aiguillage_etape
    ) , etape.chemin+[aiguillage , boucle_aiguillee] ]
38                                         })
39                     a_parcourir.add(boucle_aiguillee:aiguillage)
40     return table_tempo
41
42 #####EN Production : recalcul si on a un timer a 0 (pas de nouvelles d
    'un aiguillage) ou un message d'anomalie
43 for a in aiguillage :
44     aiguillages_timer.a = 60
45     aiguillages_timer.moi = 40
46
47 while(1):
48
```

```

49 #j'ai une anomalie
50 if (anomalie[0]) : #il y a un probleme dans ma boucle :
51     recalcule(boucle_presente , moi)
52     while(anomalie[0])
53         envoie(boucle_presente , moi)
54     recalcule("tout_va_bien", moi)
55     aiguillages_timer.moi = 12; #temporisation avant d'envoyer que tout va
    bien de nouveau
56 if (anomalie[1]) : #il y a un probleme sur mon aiguillage :
57     recalcule(boucle_aiguillee , moi)
58     while(anomalie[1])
59         envoie(boucle_aiguillee , moi)
60     recalcule(tout_va_bien , moi)
61     aiguillages_timer.moi = 12; #temporisation avant d'envoyer que tout va
    bien de nouveau
62 #tout va bien pour moi mais il faut que je le dise
63 if(aiguillages_timer.moi == 10) :
64     envoie("tout_va_bien", moi)
65     aiguillages_timer.moi = 40
66
67 #et les autres ?
68 if (recoit(anomalie , aiguillage)):
69     recalcule(anomalie , aiguillage)
70     aiguillages_timer.aiguillage = -1
71 if(recoit("tout_va_bien", aiguillage)):
72     if aiguillages_timer.aiguillage== -1: # il y a eu une erreur
73         recalcule("tout_va_bien", aiguillage)
74         aiguillages_timer.aiguillage=61
75
76 #gestion des timers
77 for a in aiguillages :
78     aiguillages_timer.a -= 1
79     if (aiguillages_timer.a==0):
80         #dans le doute on considere qu'il ne faut plus passer par l'
    aiguillage
81         recalcule("all", aiguillage)
82
83 (("--TIMER de la boucle : espacement d'1s entre chaque tour"--))
84
85
86 temp_anomalie={} # de la forme { boucle_impactee:{ aiguillage_impactant:[
    ancienne_table] } }
87
88 def recalcule(message , aiguillage):
89     table_tempo = Table
90     a_parcourir = {}
91
92     if (anomalie == "tout_va_bien"): # un probleme a ete resolu
93         for boucle , aiguillages_impactants in temp_anomalie :
94             if aiguillage , ancienne_table in aiguillage_impactants:
95                 if ancienne_table == Table_Permanente.boucle : # il n'y a pas d'
    autres alterations
96                     table_tempo.boucle = Table_Permanente.boucle
97                     #le parcours sera vide
98                 else : #il faut mieux faire un recalcul global pour cette boucle
99                     table_tempo.pop(boucle)
100                     a_parcourir.add(boucle)
101                     temp_anomalie.boucle.pop(aiguillage)

```

```

102
103 else : #il y a un probleme :
104     if (anomalie == "all"): # l'aiguillage ne donne plus de signe de vie
105         for boucle in table_tempo:
106             if boucle.chemin.contient(aiguillage):
107                 a_parcourir.add(boucle)
108                 temp_anomalie.add({ boucle:{ aiguillage:[ table_tempo.retirer(
boucle) ]}})
109     else : #message correspond a la boucle impactee
110         for boucle in table_tempo:
111             if boucle.chemin.contient(aiguillage , message):
112                 a_parcourir.add(boucle)
113                 temp_anomalie.add({ boucle:{ aiguillage:[ table_tempo.retirer(
boucle) ]}})
114 table_tempo = parcours(table_tempo , a_parcourir)
115
116 return table_tempo

```

Listing 1 – Algorithme simplifié du routage en pseudocode

Note de Cadrage et Comptes-Rendus des Réunions avec l'encadrant

Compte-Rendu 1 : mardi 2/10, réunion de lancement

(Projet Industriel) Compte-rendu Réunion de Lancement (1)

Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT et Thibault HENRY

Mardi 2 Octobre 2018

Motif de la réunion : Lancement du projet industriel	Lieu : TELECOM Nancy, salle 2.14
Présents : Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT, Thibault HENRY Quentin BILLARD, Thimothée MICHEL, Clément RI-GAUD, Sébastien HABASHI M. CHOLEZ, M. BOMBARDIER et M. MANGEOT	Date, heure de début, de fin : 02/10 de 16h à 18h

1 Ordre du jour

- Présentation globale du projet URBANLOOP par Jean-Philippe MANGEOT
- Présentation de la partie routage par Thibault CHOLEZ
- Présentation de la partie aiguillage par Vincent BOMBARDIER
- Point organisationnel

2 Discussions

2.1 Présentation globale

M. Jean-Philippe MANGEOT a pris le temps de nous expliquer plus en détail le projet URBANLOOP dans son ensemble. Il s'agit d'un projet pour remplacer en milieu urbain les transports en commun par bloc par des capsules individuelles routées suivant le trajet désiré. Il a ainsi présenté la faisabilité du projet, l'engouement des industriels et des entrepreneurs pour celui-ci, les différents terrains prévus d'expérimentation pour des tests à grande échelle, les avantages offerts par la région Grand-Est pour ce projet mené à Nancy et ses alentours, les études effectuées et les conditions de réalisation du projet comme par exemple la technologie des roues par friction la pose semi-enterrée des tubes qui dispose de nombreux avantages (un grand diamètre d'ouverture malgré un enfouissement à 90%). Cette présentation nous a montré à tous l'engouement qu'il peut y avoir pour ce projet futuriste intéressant, et auquel de nombreux acteurs vont participer. Depuis le lancement du projet en octobre 2017, de grandes avancées ont été réalisées et TELECOM Nancy va également participer à cette aventure grâce aux deux projets industriels proposés aux élèves de 3^{ème} année de l'école. Après avoir répondu à nos questions concernant le projet, M. MANGEOT a donné la parole aux encadrants universitaires pour discuter plus en détail des projets industriels.

2.2 Partie routage

2.2.1 Objectifs

Le but de ce groupe est de réaliser une application logicielle servant de maquette numérique d'un réseau urbain de capsules comme celui envisagé par l'équipe du projet URBANLOOP. Cette application simulera donc sur un réseau de boucles des capsules se déplaçant à vitesse constante et permettra de suivre l'évolution des capsules, l'état des stations de départ, et proposera quelques informations plus globales (par exemple la congestion moyenne). Cette application prendra en entrée de nombreux paramètres tels que le temps d'arrêt d'une capsule en station, ou encore la distance entre les capsules.

Une des clés de ce projet réside dans l'algorithme de routage de capsules qui va être choisi. Un état de l'art des différents algorithmes de routage devra être mené pour découvrir ce qui existe déjà, analyser ce dont nous tirerons de cette recherche et, bien entendu la mise en application d'un protocole. Ce protocole pourra être créé de toutes pièces par nos soins si le résultat de nos recherches ne s'avère pas assez satisfaisant.

De plus, l'application comprendra un simulateur d'événements discrets qui modélisera l'arrivée d'utilisateurs. Le choix du simulateur sera également déterminant pour l'application, car il impactera fortement le comportement d'une simulation. Nous essaierons de nous rapprocher de la réalité le plus possible, par exemple en utilisant les lois de probabilité adaptées à ce genre de problème.

L'objectif principal est de valider la faisabilité du projet en gérant les données en entrée et en récupérant les éléments déterminants.

2.3 Jalonnement

1. Étude fonctionnelle qu'il faudra faire en collaboration avec (et surtout à faire valider par) M. MANGEOT. Cette dernière devra regrouper tous les paramètres à prendre en compte.
2. État de l'art des différents simulateurs à événements discrets. Il faudra déterminer si nous utiliserons un simulateur existant ou si nous en développerons un *from scratch*).
3. Démarrage de l'implémentation de l'application. À commencer aux environs de décembre, lorsque les choix du simulateur et de l'algorithme de routage seront effectués.

2.3.1 Livrables attendus

- Note de cadrage : première ébauche des besoins fonctionnels.
- Documentation fonctionnelle (cahier des charges).
- Rapport intermédiaire : contenant le cahier des charges et l'état de l'art.
- Soutenance intermédiaire (décembre) : présentation du cahier des charges et de l'avancement.
- Documentation technique.
- Rapport final.
- Soutenance finale.
- Sources du code, commentées et exploitables.
- Article pour promouvoir notre travail sur le site URBANLOOP : vulgarisation du travail et présentation des résultats, animations... Le tout accompagné de tous les éléments créatifs (*kakemono*, vidéos...) pour mettre en avant le projet.
- Animation visuelle et interactive de la simulation.

2.3.2 Établissement des premiers éléments à prendre en compte

Il faudra considérer plusieurs éléments et paramètres que nous avons déjà succinctement énumérés. Cette liste n'est pas exhaustive et peut être sujette à changements.

- Le flux principal ;
- La capacité des stations ;
- Le modèle actuel accepte que la majorité des capsules dans le réseau soit vide, notre modèle pourra (et devra) donc être considéré de la sorte ;
- La file d'attente dans chaque station avant de rentrer dans une capsule ;
- La phase d'accélération (concentration sur la durée et non considération de la distance) ;
- L'existence de centres de stockage des capsules ;
- Gérer un flux important (heure de pointe) ;
- Gérer les intersections (changement de boucle d'une capsule et entrée dans le réseau) ;
- Possibilité de mettre en ligne le simulateur (optionnel).

Paramètres	Modélisation	Valeur
durée d'accélération	Paramètre	6s
distance de garde	Paramètre	50cm
vitesse	constante	80km/h
apparition des utilisateurs	fréquence variable avec trajets aléatoires	loi de poisson + gestion des "heures de pointe"
file d'attente en station	varie suivant les stations	provision à moduler (4 emplacements, minimum 2 capsules)
temps de sortie		Gaussienne centrée en 8s

2.4 Partie aiguillage

Cette partie étant dédiée aux élèves de l'autre groupe du projet URBANLOOP, il ne sera pas nécessaire d'en effectuer une description complète dans ce compte-rendu. Ce sujet concerne l'insertion des capsules de transport dans une boucle du réseau et la réalisation d'un changement de boucle. À cela s'ajoute la gestion de l'accélération et la décélération des capsules pour éviter toute collision et créer la place nécessaire à l'insertion d'une capsule. Le groupe devra produire une maquette physique en utilisant des locomotives ou voitures communicantes.

Des liens potentiels entre les deux projets ont été évoqués, notamment le fait que le résultat final et les solutions envisagées par l'autre groupe pourraient être implantés dans notre simulateur d'un réseau URBANLOOP en toute fin de réalisation.

2.5 Organisation

Nous avons mis en place ou allons mettre en place différents éléments :

- Slack (plateforme de discussion)
- Agenda Google pour notifier tous les événements importants
- Drive d'équipe Google (dépôt des éléments de gestion de projet)
- Dépôt GitHub (à initialiser)
- Gantt (à définir)

Il a aussi été notifié que le travail devra se faire en autonomie (il ne faut pas hésiter à énoncer des hypothèses concernant la réalisation du simulateur). Néanmoins il sera nécessaire de tenir informer les encadrants des différentes avancées majeures, et surtout faire un point hebdomadaire avec M. CHOLEZ de notre avancement.

3 Actions à suivre

Quoi ?	Qui ?	Délai ?
Compte-rendu de lancement	Charlotte, Baptiste et Thibault	05/10
Premier jet de la note de cadrage	Charlotte, Baptiste et Thibault	10/10
Donner les droits d'accès à M. Cholez	Charlotte, Baptiste et Thibault	10/10
Note de cadrage	Charlotte, Baptiste et Thibault	13/10

Date de la prochaine réunion : Mardi 10 octobre 2018, 14h

Note de Cadrage

PROJET URBANLOOP, ROUTAGE

SIMULATION DU RÉSEAU DE TRANSPORT URBAIN PAR CAPSULES URBANLOOP

Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT & Thibault HENRY
mardi 9 octobre 2018

Acteurs du Projets :

Demandeur (et encadrant industriel) :

Jean-Philippe MANGEOT jean-philippe.mangeot@univ-lorraine.fr

Encadrant Académique (et encadrant industriel privilégié) :

Thibault CHOLEZ thibault.cholez@inria.fr

Équipe du projet :

Charlotte ASSELIN-BOULLÉ charlotte.asselin-bouille@telecomnancy.eu

Baptiste CHOCOT baptiste.chocot@telecomnancy.eu

Thibault HENRY thibault.henry@telecomnancy.eu

Énoncé du Projet :

Intitulé :

Simulation du réseau de transport urbain par capsules Urbanloop

Entitled :

Simulation of the capsule-based urban transportation infrastructure Urbanloop

Contexte

L'Urbanloop s'inscrit dans le cadre d'un souhait de remaniement des transports urbains. L'objectif est de proposer un réseau cheminant des capsules individuelles lancées à grande vitesse. Le projet global comprend une collaboration inter-écoles et une mise en place dans l'agglomération Nancéenne où un réseau de test verra le jour. Le projet ci-après présenté se concentre sur l'aspect de création d'une maquette numérique simulant le réseau, sans tenir compte outre mesure des autres projets collaborateurs.

Objectifs globaux

Ce projet vise à établir la faisabilité du projet global du point de vue des dimensionnements, du routage, et plus largement dans sa globalité. Il s'agira de mettre en place un simulateur numérique avec adaptation des différentes données d'entrée considérées comme les paramètres. Il faudra par ailleurs pouvoir récupérer les éléments déterminants, quantifiés. Il sera aussi nécessaire de proposer une interface graphique interactive de cette simulation pour promouvoir le projet.

Attendus des parties prenantes

Objectifs pour les demandeurs

Comme explicité dans les objectifs principaux du projet, il s'agit de déterminer la faisabilité du réseau Urbanloop pour valider ou non l'établissement du réseau.

La maquette, avec une interface utilisateur interactive pourra aussi servir de support pour d'éventuelles demandes de subventions. La simulation servira ainsi de validation aux équipes, mais aussi d'outil pour déterminer le dimensionnement ou encore le système de routage utilisé.

Objectifs de l'équipe

Pour l'équipe en charge de l'élaboration de la maquette c'est l'occasion de découvrir les projets industriels en se confrontant aux contraintes des projets moins scolaires et plus concrets. Cette étude permet aussi de s'investir dans un projet innovant et de le porter.

Besoins Fonctionnels que devra satisfaire le projet

Les besoins sont listés par ordre de priorité :

1. Réalisation d'une maquette numérique (ou simulateur) réaliste du réseau de transport Urbanloop avec visualisation unitaire des capsules et usagers transitant en temps réel et permettant d'estimer le débit maximum admissible du réseau avant congestion en fonction de plusieurs paramètres.
2. Conception de l'algorithme de routage des capsules optimisant le temps de transport.
3. Modulation de la maquette numérique via différents paramètres à inclure. Le réseau sera modélisé par un simulateur à événements discrets incluant :
 - Le nombre de capsules et d'usagers.
 - La topologie du réseau (nombre de boucles, d'inter-connections, de stations et de compartiments par station)
 - Les différents délais (temps d'arrêt des capsules en station (montée et descente des usagers), temps d'insertion dans le réseau (correspondant au temps d'accélération)).
 - La vitesse (constante) dans le circuit et la distance de sécurité inter-capsules.
 - Les usagers : il faudra considérer une arrivée probabilistes des usagers avec des souhaits de trajets pseudo-aléatoires. Le modèle devra aussi simuler les flux journaliers des usagers avec la notion d'heure de pointe : une arrivée massive avec des destinations semblables.
 - La distance inter-capsules pour assurer la sécurité tout en exploitant au maximum le circuit.
4. Inclure une interface de visualisation en "temps réel" et interactive.
5. Créer un algorithme de stockage et d'approvisionnement des capsules pour anticiper les demandes des stations suivant les flux journaliers.

Livrables attendus

- 12/10 - Note de cadrage : première ébauche des besoins fonctionnels (document ci-présent).
- 17/12 - Rapport intermédiaire concluant la phase d'étude : cahier des charges, état de l'art des simulateurs et choix technologiques, spécification des algorithmes.
- 20/12 - Soutenance intermédiaire : présentation de la phase d'étude
- 15/03 - Rapport final présentant les phases d'étude, conception et réalisation du projet.
- 15/03 - Sources du code, commentées et exploitables (open-sources).
- 15/03 - Documentation technique, manuel d'utilisation de l'outil.
- 15/03 - Article pour promouvoir notre travail sur le site Urbanloop : vulgarisation du travail et présentation des résultats, animations... Le tout accompagné de tous les éléments créatifs (*kakemono*, vidéos...) pour mettre en avant le projet.
- 15/03 - Animation visuelle et interactive de la simulation.
- 19/03 - Soutenance finale.

Organisation

Nous avons mis en place différents éléments organisationnels dans le but de mener à bien ce projet et la communication entre les membres de l'équipe :

- Slack (plateforme de discussion)
- Agenda Google lié à Slack pour notifier tous les événements importants (réunion, rendus, jalons...)
- Drive d'équipe Google (dépôt des éléments de gestion de projet)
- Dépôt GitHub (dépôt du code source du simulateur)
- Un jalonnement défini via un diagramme de Gantt (en annexe de ce document) que nous ajusterons au fur et à mesure du projet. Le projet sera constitué de 3 parties majeures : La partie étude des besoins fonctionnels, ensuite l'élaboration de la maquette (conception des algorithmes de simulations et de routage et enfin l'implémentation (du simulateur avec l'aspect routage puis de l'interface utilisateur).

Moyens mis à dispositions et Contraintes associées

Humains : Notre équipe est composée de trois membres, l'un issu de l'approfondissement IL (Ingénierie du logiciel) et deux de l'approfondissement ISS (Internet, Systèmes connectés et Sécurité) de TELECOM Nancy. Thibault CHOLEZ et Jean-Phillipe MANGEOT seront les encadrants de ce projet et c'est principalement avec eux que nous échangerons pour tout renseignement concernant le projet.

Temporels : Pour réaliser ce projet industriel, nous disposerons de 250 heures de travail par élève soit un total de 750 heures cumulées. Elles seront réparties sur les 25 semaines de durée impartie à ce dernier, dont une semaine banalisée pour l'avancé du projet et de 4 semaines de vacances scolaires. Ces heures devront être assignées de manière coordonnée pour mener à bien le planning prévisionnel établi (ci-joint le diagramme de Gantt en annexe). Il faut toutefois noter que des contraintes temporelles pourraient s'ajouter quant aux emplois du temps scolaires respectifs de chacun d'entre nous.

Matériels : Aucun matériel précis sera requis pour ce projet industriel mis à part nos ordinateurs et l'achat de licences de logiciel si nécessaire.

Risques globaux : Matrice des risques selon leur probabilité et l'incidence sur le déroulé du projet :

Incident	Probabilité	Criticité	Actions de compensation
Maladie d'un membre	1	1	Répartir les tâches affectées à ce membre aux deux personnes restantes
Problème de délai	3	2	Modifier le Gantt afin de respecter au mieux les dates de rendu de livrable
Blocage de l'équipe	2	3	Établir une réunion entre les membres de l'équipe et les encadrants pour discuter des différents problèmes rencontrés afin de trouver des solutions adaptées
Algorithmes erronés	2	3	Si les résultats escomptés ne sont pas atteints, il sera nécessaire de se regrouper pour comprendre les erreurs de l'algorithme et les corriger
Aucun simulateur existant adapté pour le projet	2	1	Produire notre propre simulateur ou décider de migrer vers un autre langage pour explorer d'autres possibilités

Lu et approuvé le jeudi 11 octobre 2018

Équipe projet

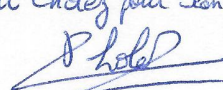
Encadrant industriel

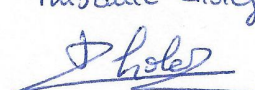
Encadrant universitaire

ASSELIN-BOULLE Charlotte

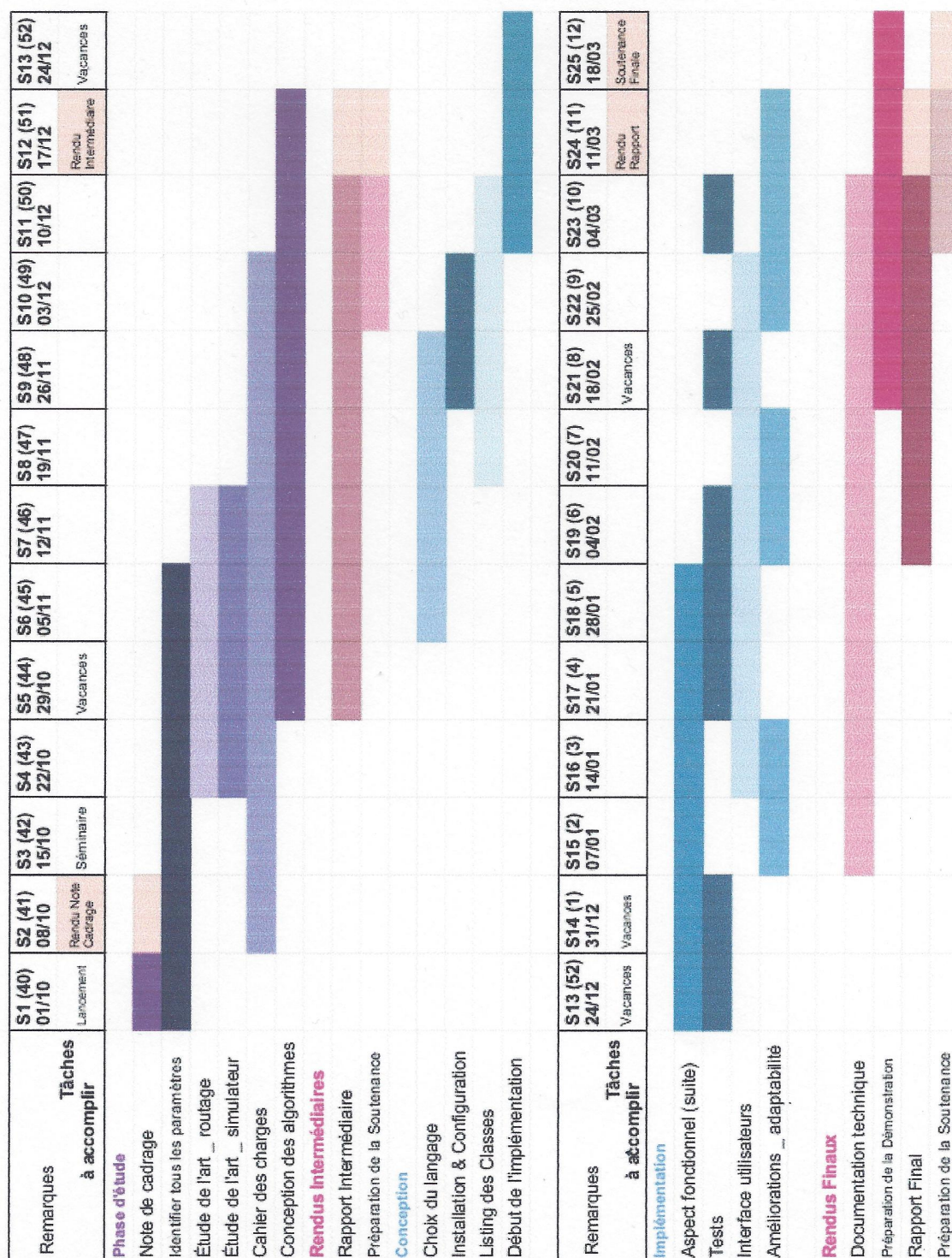
HENRY Thibault

CHOCOT Baptiste


Thibault Cholez pour Jean-Philippe Mangeot


Thibault Cholez


Annexe : Diagramme de GANTT prévisionnel



Compte-Rendu des points durant la semaine banalisée, du 14 au 18 Janvier

(Projet Industriel) Compte-rendu des différents points durant la semaine banalisée

Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT et Thibault HENRY

Du Lundi 14 au Vendredi 18 Janvier 2019

Différents points

- Lundi 14 Janvier : organisation de la semaine au sein du groupe
- Mardi 15 Janvier : rencontre avec Monsieur MANGIN et présentation du travail effectué jusque là.
- Mercredi 16 Janvier : point d'avancement, d'organisation et retours sur la réunion précédente avec Monsieur CHOLEZ
- Mercredi 16 Janvier : point d'avancement avec Monsieur MANGEOT
- Vendredi 18 Janvier : retour sur la semaine avec monsieur CHOLEZ

1 Discussions

1.1 Organisation de la semaine

Motif de la réunion : Organisation de la semaine	Lieu : TELECOM Nancy, S0.11
Présents : Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT et Thibault HENRY	Date, heure de début, de fin : 14/01 de 9h à 9h30

1.1.1 Échange :

Les étudiants ont rappelé et redéfini les objectifs et la répartition de la semaine.

1.1.2 Éléments pour la suite :

L'objectif est d'avoir une première version fonctionnelle de notre application.

- Charlotte s'occupe du routage et de l'établissement d'un réseau ;
- Baptiste se concentre essentiellement sur l'interface graphique ;
- Thibault configure le simulateur et les différents éléments associés.

1.2 Rencontre avec monsieur MANGIN

Motif de la réunion : Rencontrer M. MANGIN, mise au point sur le projet et présentation du travail effectué jusqu'alors	Lieu : TELECOM Nancy, S2.40 (salle du conseil)
Présents : Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT et Thibault HENRY, Philippe MANGIN, Vincent BOMBARDIER, 5 élèves des PIDR	Date, heure de début, de fin : 15/01 de 15h45 à 17h

1.2.1 Échange :

Dans un premier temps, nous avons présenté le travail réalisé et les objectifs (à court et long termes) de notre partie du projet. Nous avons expliqué que, pour l'instant, nous nous sommes essentiellement consacrés à l'étude, la spécification et la conception de notre projet et que nous débutons seulement l'implémentation. Nous avons pour objectif d'avoir une version fonctionnelle basique au plus vite puis de procéder par jalons.

Monsieur MANGIN nous a ensuite expliqué son rôle dans le projet global : faire en sorte d'harmoniser l'ensemble du projet en donnant une vision d'ensemble à chacun des partis. Il a insisté sur l'intérêt qu'il portait à notre projet, qui, par son visuel, permettra de faire comprendre mais aussi convaincre de nombreux acteurs ou potentiels investisseurs. Il s'agit en effet de pouvoir voir l'un des aspects innovants du projet global : la puissance organisatrice derrière le réseau. Il espère aussi que nous pourrons ainsi tester les cas limites de saturation du réseau et pouvoir effectivement prouver la fiabilité et la sûreté de fonctionnement du projet global.

1.2.2 Éléments pour la suite :

Il faut échanger avec monsieur MANGIN pour qu'il nous guide vers une uniformisation des projets.

Il est important d'avoir un visuel de l'application assez rapidement.

Une rencontre de tous les acteurs du projet aura lieu le jeudi 7 Février après-midi. Il faudra alors préparer quelques slides (en anglais) et quelques mots de présentation (en français) pour présenter dans quel aspect on s'inscrit dans le projet, nos objectifs, etc. Ces derniers serviront d'archivage sur un Wiki qui regroupera les acteurs du projet pour permettre une vue d'ensemble à tous, d'inter-connecter et de déterminer les orientations des collaborateurs.

1.3 Point avec Monsieur CHOLEZ

Motif de la réunion : Retour sur la réunion de la veille, état de l'avancement, organisation	Lieu : TELECOM Nancy, S0.11
Présents : Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT, Thibault HENRY et Thibault CHOLEZ	Date, heure de début, de fin : 16/01 de 10h à 11h

1.3.1 Échange :

Nous avons fait un retour sur la réunion de la veille avec Monsieur CHOLEZ et en particulier avons vérifié qu'il ne s'agissait pas simplement d'avoir une interface sans aspects algorithmiques viables implémentés comme monsieur MANGIN nous l'a proposé.

Nous avons aussi parler de l'objectif des étudiants de PIDR et les liens entre nos projets : ils reprendront ce que nous avons effectué en essayant d'y inclure leur partie (routage SDN) pour déterminer si les limites sont plus intéressantes. Nous avons ensuite présenté le travail actuel : un routage dynamique fonctionnel même si non encore optimal, un début de simulateur à événement discret notamment l'apparition des usagers des transports selon une répartition probabiliste la plus réaliste possible. Enfin, un début d'interface graphique avec l'explication de ce qu'il est prévu d'en faire.

1.3.2 Éléments pour la suite :

Continuer à aller vers le réseau de test pour permettre de tester nos parties sur le même projet. En particulier, s'accorder sur l'aspect de stockage des positions des boucles et capsules. Privilégier les angles pour permettre des zooms et dézooms. Pour l'apparition des voyageurs, rajouter des attributs et la création d'un pattern suivant les stations pour déterminer suivant les heures où les voyageurs sont le plus susceptibles de vouloir partir et arriver. Commencer au plus vite à fusionner nos différentes parties.

1.4 Point avec Monsieur MANGEOT

Motif de la réunion : Visite de Monsieur MANGEOT pour se tenir informé de l'avancement	Lieu : TELECOM Nancy, S0.11
Présents : Charlotte ASSELIN-BOULLÉ, Thibault HENRY et Jean-Philippe MANGEOT	Date, heure de début, de fin : 16/01 de 16h à 17h

1.4.1 Échange :

Monsieur MANGEOT est venu voir notre avancement sur le projet et le respect de nos objectifs. Il a insisté sur le fait qu'il n'est pas intéressé par des versions fonctionnelles non complètes par rapport aux objectifs qu'il a fixé. En outre, il souhaiterait que l'aspect optionnel défini dans la note de cadrage soit effectif, à savoir la gestion des slots au sein des stations et donc l'absence de "capacité infinie".

1.5 Retour sur la semaine avec Monsieur CHOLEZ

Motif de la réunion : Récapitulatif de la semaine et organisation future	Lieu : TELECOM Nancy, S0.11
Présents : Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT, Thibault HENRY et Thibault CHOLEZ	Date, heure de début, de fin : 18/01 de 15h50 à 16h20

1.5.1 Échange :

Monsieur CHOLEZ a souhaité vers un rapide retour sur notre expérience de la semaine et le respect ou non des objectifs. Finalement, nous n'avons pas réussi à livrer une version fonctionnelle du projet. En effet, nous nous sommes trop dissipés par rapport à la tâche centrale. En particulier, chacun des éléments du groupe a bien avancé dans les détails de son implémentation et les 3 parties sont fonctionnelles mais il n'y a pas eu de mise en commun suffisamment efficace pour avoir une application globale fonctionnelle.

1.5.2 Éléments pour la suite :

Procéder par sprints bien définis. Finir cette version 1 au plus vite.
S'accorder plus entre nous pour pouvoir fusionner nos parties.

2 Actions à suivre pour la suite

Sprint du 19/01 au 29/01

Quoi ?	Qui ?	Délai ?
Amélioration Routage permanent (coût)	Charlotte	mar 22/01
Lien entre modèle et interface	Baptiste	mar 22/01
Monter dans la capsule départ d'un capsule	Thibault	dim 27/01
Trajet d'une capsule	Thibault , Charlotte	dim 27/01
Arrivée et descente d'un capsule	Thibault	dim 27/01
Positionnement intelligent des loops	Baptiste	dim 27/01
CR et gestion de l'administratif	Charlotte , Thibault et Baptiste	lun 28/01

Objectifs :

- Avoir un routage statique fonctionnel et optimal
- Un interface qui visualise le modèle chargé en s'adaptant (*mini_network* pour l'implémentation, *routing_capsule* pour la position des capsules)
- Un simulateur qui gère la liste des événements

Tâches déjà visualisées à prévoir pour les sprints suivants :

Quoi ?	Qui ?
Interface - Positionnement des capsules	Baptiste , Thibault
Routage dynamique	Charlotte
Défauts	Thibault , Charlotte
Affichage des informations	Baptiste
Boutons de gestion de simulation (temps)	Baptiste
Zoom, dézoom et déplacement	Baptiste

Date de la prochaine réunion : Mardi 18 décembre 2018, 14h

Compte-Rendu de retro-planning du 12 Février

(Projet Industriel) Compte-rendu Réunion RetroPlanning (9)

Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT et Thibault HENRY

Mardi 12 Février 2019

Motif de la réunion : Point sur l'Avancement et Retro-Planning	Lieu : TELECOM Nancy, salle 2.30
Présents : Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT, Thibault HENRY, M. CHOLEZ	Date, heure de début, de fin : 12/02 de 14h à 15h

1 Ordre du jour

- Retour sur l'avancement
- Rétro Planning par rapport aux objectif décrits dans la note de cadrage

2 Retour sur l'avancement

Suite au tournage de la vidéo de mardi 5 et la rencontre de tous les acteurs du jeudi 7, nous faisons avec Monsieur CHOLEZ le point sur notre avancée.

Nous avons actuellement un simulateur fonctionnel et nous pouvons suivre les trajets via les logs (DEBUG et INFO suivant la quantité d'information souhaitée). L'interface graphique n'est pas encore complètement opérationnelle : nous avons des capsules qui se "téléportent".

Pour remédier aux incohérences de l'interface, nous avons fait une queues des états du réseau et la vue va chercher les éléments du tics à sa vitesse. Le problème actuel de ce fonctionnement est qu'il va y avoir un léger laps de temps avant la prise en compte d'un défaut ce qui correspondra à quelques secondes.

Nous avons aussi rajouté le fait que des capsules vides peuvent partir des stations "trop remplies" pour homogénéiser la répartition des capsules dans les stations. De plus, nous prenons en compte la capacité d'une station : si une capsule souhaite entrer dans une station déjà pleine, elle "refait un tour" et cet événement déclenche le drainage de la station pleine. Nous avons prévu que ce "drainage" soit déclenché régulièrement et pas seulement quand une capsule ne peut pas entrer dans une station.

3 Retro Panning

Il reste 4 semaines et demi avant le rendu final du projet :

- La fin de la semaine actuelle (mardi 12 février)
- La semaine des vacances (mardi 19 février)
- La semaine de la rentrée (prochaine réunion mardi 26 février)
- Le semaine banalisée (du 4 au 8 mars, mardi 5)
- La semaine des jeux d'entreprise, relativement bloquée. Avec le rendu du rapport le vendredi 15 mars (mardi 12 mars)
- La soutenance le mardi 19.

Nous avons repris la note de cadrage pour déterminer les éléments déjà effectués et ceux qu'il reste à fixer et/ou à faire.

Conformément aux objectifs, nous avons déjà :

- un simulateur basique fonctionnel
- topologie complète et modulable
- solution adaptative et paramétrable
- tenir compte des délais

Il reste différents éléments à inclure notamment :

- récupérer des statistiques
- fixer l'interface graphique pour une visualisation unitaire des capsules et usagers
- rajouter la visualisation des queues (usagers et capsules (slots)) des stations
- ajouter un menu de paramétrage
- gérer l'approvisionnement : nombre de capsules fixes et création des entrepôts
- Vérifier les collisions - voir avec l'autre groupe pour l'homogénéisation avec / l'incorporation de leur travail

Nous avons donc profité de ce listing pour effectuer une première répartition de ces tâches. À noter que nous avons aussi parlé du rapport final qui devra reprendre le rapport intermédiaire en le complétant (si besoin nous basculerons des parties dans les annexes). Il faudra ainsi avoir un plan sous la forme Conception (la majeure partie du rapport intermédiaire), Réalisation (notre résultat abouti), Évaluation (contenant les différents tests, statistiques, limites observées... et pour chaque objectif de la note de cadrage, un test unitaire montrant la vérification de ce point).

Nous avons aussi établi qu'il faudra planifier un test d'acceptation avec Monsieur MANGEOT et/ou Monsieur MANGIN la semaine de la soutenance.

4 Actions à suivre

Quoi ?	Qui ?	Délai ?
Gérer la descente des usagers	Thibault	V.15/02
Rajouter vue de paramétrage	Baptiste et Charlotte	Ma.19/02
Fixer le nombre de capsules	Charlotte	J.28/02
Visualisation unitaire des capsules et usagers	Baptiste	Ma.26/02
Présence d'entrepôts - approvisionnement	Charlotte et Thibault	Me.6/03
Gestion des défauts et collision	Thibault et Charlotte	V.8/03
Interface pleinement adaptative (topologie)	Baptiste	V. 8/03
Visualisation des queues dans les capsules	*	V.8/03
Récupération des statistiques	Charlotte et Thibault	ASAP
Résumé	*	envoi V. 15/03
Rapport	*	V. 15/03
Présentation soutenance	*	Ma. 19/03

Date de la prochaine réunion : Mardi 26 Février 2019 à 14h

Résumé

Ce document expose les différents travaux de recherches effectués dans le cadre du projet industriel : *Simulation du réseau de transport urbain par capsules URBANLOOP*. Ce projet s'inscrit dans le cadre d'une ambition de remaniement des transports urbains en proposant un réseau cheminant des capsules individuelles. Ce document et notre projet se concentrent uniquement sur l'aspect de création d'une maquette numérique simulant le réseau. Nous présentons dans ce dossier l'ensemble de notre travail : le résumé des phases de spécification et conception, la réalisation et l'implémentation, ainsi qu'une évaluation des objectifs initiaux.

Mots-clés : réseau de transport, routage, simulateur à événements discrets

Abstract

This document thoroughly exhibits the research work carried out as part of the industrial project : *Simulation of urban transport network by URBANLOOP capsules*. This project falls within the framework of a reshuffling ambition of urban transports by proposing a network of individual capsules. This paper and our project are only focusing the creation of a digital model simulating the network. We present in this document the specification, the design, and the implementation phases, as well as an assessment of the initial objectives. **Keywords : transport network, routing, discrete event simulator**